
Communication-Efficient LLM Adaptation over Decentralized GPU Meshes

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Decentralized training enables large-model training over low-end GPUs and
2 internet-grade connections, but communication along both data-parallel and
3 pipeline-parallel axes becomes the primary bottleneck. We study post-pretraining
4 adaptation in this setting. We propose an asynchronous two-circuit system: a
5 fast compressed training circuit drives throughput using activation masking for
6 pipeline-parallel (PP) transfer and compressed data-parallel (DP) synchronization,
7 while a slow anchor circuit runs occasional unmasked forward-backward passes off
8 the critical path. Then, we introduce a spectral correction optimizer that uses these
9 delayed *anchor priors* to denoise masked gradients without blocking the fast stream.
10 Although prior work has found aggressive activation compression unreliable, we
11 show that masking supports post-pretraining adaptation at high compression rates
12 when anchored this way. Pipeline-parallel compression alone yields up to a $9\times$
13 throughput gain, and combining it with data-parallel compression increases be-
14 yond $40\times$ over internet-grade $\sim 200\text{Mbps}$ connections, while matching dense
15 uncompressed performance across domain adaptation and continual pretraining.

16 1 Introduction

17 Training and adapting frontier models typically requires tightly connected datacenter clusters, putting
18 modern AI research out of reach for smaller labs, independent researchers, and open communities
19 [Narayanan et al., 2021, Grattafiori et al., 2024]. Decentralized training offers an alternative: com-
20 putation can be distributed over the Internet across heterogeneous consumer-grade GPUs, allowing
21 participants to contribute otherwise fragmented compute [Diskin et al., 2021, Ryabinin et al., 2023,
22 Yuan et al., 2022, Douillard et al., 2023a, Jaghouar et al., 2024a, Ajanthan et al., 2026, Ramas-
23 inghe et al., b]. We focus on decentralized post-pretraining adaptation, *i.e.*, fine-tuning or continual
24 pretraining open-weight models over such GPU meshes.

25 This setting is fundamentally communication-limited. Commodity Internet links provide bandwidth
26 on the order of hundreds of Mbps, compared to hundreds of Gbps in datacenter interconnects. A
27 practical system must therefore reduce communication along both distributed-training axes: data
28 parallelism (DP), where replicas synchronize weight gradients, and pipeline parallelism (PP), where
29 model shards exchange activations and activation gradients.

30 DP communication can be reduced with gradient compression [Vogels et al., 2019, Wang et al.,
31 2018, Basu et al., 2019, Zhao et al., 2024a] or sparse synchronization [Douillard et al., 2023b, 2025,
32 Peng et al., 2024, Jaghouar et al., 2024b, Sani et al., 2024], but these methods still assume that each
33 worker or replica group can host a full model. PP addresses the complementary regime: it enables
34 models that do *not* fit on a single low-end GPU by partitioning them across layers. However, PP
35 compression is substantially more delicate. Standard compression methods such as sparsification,
36 quantization, and low-rank projection often degrade performance at high compression rates due to

error accumulation across layers [Rudakov et al., 2023, Ramasinghe et al., 2025]. Recent methods such as Subspace Networks [Ramasinghe et al., 2025] and Pufferfish [Wang et al., 2021] mitigate this during pretraining by imposing structural constraints, but these constraints make them difficult to apply to arbitrary pretrained checkpoints.

We therefore revisit and refine a simple but previously discarded idea: *activation masking*. Our key observation is that adaptation differs from pretraining. During post-pretraining adaptation, weights evolve slowly from a strong pretrained initialization, and the goal is often to find a useful update direction rather than learn representations from scratch. In contrast to the pessimistic view in prior works [Ramasinghe et al., 2025, Rudakov et al., 2023], we show that sparse activation signals are sufficient for the main training stream, provided they are anchored by higher-fidelity information. We realize this idea with a two-circuit decentralized system. A fast compressed circuit performs most updates using in-graph activation masking along the PP axis and compressed synchronization along the DP axis. In parallel, a slower anchor circuit occasionally runs full unmasked forward and backward passes. These unmasked passes do not block the fast circuit; instead, their gradients arrive asynchronously as *Anchor Priors*, which denoise and steer the masked optimization trajectory. The asynchrony is principled: fine-tuning gradients live in a low-dimensional [Li et al., 2018, Aghajanyan et al., 2021, Gur-Ari et al., 2018], slowly-drifting subspace, so a stale unmasked gradient still points roughly the right way [Neyshabur et al., 2020, Wortsman et al., 2022]. This is akin to the same structural property that makes model merging effective [Wortsman et al., 2022], applied here at the gradient level.

To use these delayed priors effectively, we introduce a spectral correction optimizer. It maintains a slow-moving momentum buffer over anchor gradients and uses its principal directions to reweight masked updates: anchor-supported eigendirections are reinforced, while unsupported noisy directions are damped. We complement this design with a convergence analysis identifying when the correction provably helps: when fine-tuning gradients lie in a low-dimensional subspace and the optimization trajectory drifts slowly, the anchor basis remains informative despite staleness. The analysis also clarifies why random PRF masking is essential; it produces an approximately unbiased gradient estimator that the spectral filter can denoise, while top- K masking introduces a structured bias the filter cannot remove. Empirically, the system is robust to substantial staleness, and significantly superior to top- K , reinforcing these predictions.

Across domain adaptation, continual pretraining, and reasoning-oriented training, our method matches or exceeds dense uncompressed training while substantially improving throughput over low-bandwidth links. PP compression alone yields up to a $9\times$ throughput gain, and combining it with DP compression increases this over $40\times$. The method generalizes across model families, depths, widths, and mesh sizes, and produces models that are more robust to pruning. Overall, our contributions are as follows:

- We propose a two-circuit decentralized training system in which a fast masked circuit trains continuously while a slower unmasked anchor circuit asynchronously supplies high-fidelity gradient priors, decoupling throughput from the cost of clean gradient signals.
- We introduce a spectral correction optimizer that uses delayed Anchor Priors to denoise masked gradients. Our convergence analysis and empirical results demonstrate its efficacy.
- We show superior performance among competing methods, matching uncompressed adaptation while achieving over a $40\times$ throughput. Further, models trained with our method are more robust to pruning, improving their suitability for resource-constrained inference.

2 Related Works

Data parallel compression. Decentralized DP training has motivated two lines of work: 1) *Sparse synchronization methods* reduce the number of all-reduces by performing many local optimizer steps between outer updates [Douillard et al., 2023b, 2025, Peng et al., 2024, Jaghouar et al., 2024b, Sani et al., 2024]; we use Streaming DiLoCo [Douillard et al., 2025], which overlaps the outer synchronization with local computation. 2) *Low-rank gradient compressors* reduce communication volume by transmitting low-rank approximations of the gradient [Vogels et al., 2019, Wang et al., 2018, Basu et al., 2019, Zhao et al., 2024a]; we combine PowerSGD [Vogels et al., 2019] with Streaming DiLoCo. However, the proposed design can be integrated with other DP alternatives.

Pipeline-parallel activation compression. *Quantization-based methods* cast activations and activation gradients to lower-precision formats with error compensation: AQ-SGD [Wang et al., 2022a] compresses activation *changes* between epochs to enable error feedback over slow networks, and TAH-Quant [He et al., 2025] extends this with tile-wise adaptive Hadamard quantization. These methods operate *off-graph*, transmitting compressed payloads that are decompressed before use, which decouples forward and backward sparsity patterns and requires storing per-example state across epochs to support error feedback. We instead use *in-graph* random masking with no error feedback. Our masking is also substantially more aggressive (95% vs. $4 \times 8 \times$ quantization). **Parameter-efficient fine-tuning.** reduces training VRAM by using low-rank adapters [Hu et al., 2022, Zhao et al., 2024b, Lialin et al., 2023, Li and Liang, 2021, Lester et al., 2021, Houlisby et al., 2019, Pfeiffer et al., 2021], but the forward and backward passes still transmit *full* dense activations across pipeline-parallel boundaries. Thus, they do not address the communication bottleneck. Our method composes cleanly with PEFT (Section 6).

3 Methodology

We consider decentralized post-pretraining adaptation over a low-bandwidth GPU mesh across DP and PP dimensions. We represent a decentralized mesh as a 2D $X \times Y$ grid, where X is the data-parallel (DP) dimension and Y is the pipeline-parallel (PP) dimension. Each worker stores only a subset of the model layers, and workers communicate over low-bandwidth links along both DP and PP axes. For all experiments, we partition the mesh into two asynchronous circuits: an $(X - Z) \times Y$ fast masked circuit and a $Z \times Y$ unmasked *anchor circuit*. In all our experiments, we fix $Z = 1$.

The anchor circuit occasionally runs unmasked forward-backward passes and supplies high-fidelity gradient signals that the fast circuit uses to denoise its compressed updates. The anchor circuit does not block the fast circuit: its gradients are consumed asynchronously when they arrive, so the fast circuit’s throughput is set by masked communication only. This split is the core design move. Activation masking alone reduces communication but introduces noise that compounds across boundaries; running unmasked passes synchronously would eliminate the throughput gain. Asynchronous anchors give us both, *i.e.*, masked-step throughput, plus a slow correctional signal that does not enter the critical path.

3.1 Activation Masking for PP Compression

Let $h \in \mathbb{R}^{T \times H}$ denote the hidden-state tensor at a masked inter-layer boundary, where T is sequence length and H is hidden size. On masked steps we apply elementwise masking $\tilde{h} = h \odot m$ with $m \in \{0, 1\}^{T \times H}$ at masking probability p , retaining $K = \text{round}((1 - p)H)$ dimensions per token.

Shared PRF avoids transmitting the mask. Naively, masking would require sending the retained indices for every token alongside the values, halving the compression benefit. We avoid this by generating m from a shared deterministic pseudorandom function $M = g(\ell, t, B, S, H)$ keyed on layer index, optimization step, and batch metadata. Sender and receiver reconstruct the same mask locally; only the K retained values are transmitted.

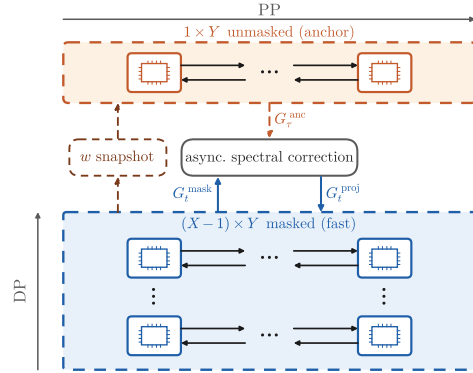


Figure 1: **System overview.** A small unmasked anchor circuit (orange, $1 \times Y$ PP stages) asynchronously supplies clean gradients G_t^{anc} to a spectral corrector, which denoises the masked gradients G_t^{mask} from the fast circuit (blue, $(X - 1) \times Y$ DP $\times$ PP) into corrected updates G_t^{proj} . Slow circuit periodically pulls fresh weight snapshots from the fast circuit.

In-graph masking gives matched forward/backward sparsity. Because masking is applied in-graph as a multiplication, the backward pass inherits the same pattern automatically: $\partial\mathcal{L}/\partial h_{t,j} = (\partial\mathcal{L}/\partial \tilde{h}_{t,j}) m_{t,j}$. Any entry masked in the forward has exactly zero backward gradient, so the same retained positions can be reused for backward activation-gradient transmission. This gives the same compression rate in both directions and avoids the need to track separate forward and backward sparsity patterns. It also distinguishes our scheme from off-graph transport compression, where forward and backward compression are decoupled and can drift out of correspondence with the derivative of the executed computation (e.g., decoupled Top- K or quantization). Note that We deliberately use random masking instead top- K because 1) top- K is biased by construction, and as we show, this bias propagates through the descent inequality as a non-vanishing floor, and 2) the spectral correction we introduce next contracts zero-mean isotropic noise but cannot remove a structured bias (see Appendix F for a full analysis).

3.2 Anchor Priors and Spectral Correction

Activation masking reduces PP communication but injects noise into every gradient. We need a denoising signal, but one that does not eliminate the throughput gains we just bought. Fine-tuning makes a useful option viable: gradients live in a slowly drifting low-dimensional subspace, so a stale unmasked gradient still carries useful information. We exploit this with an asynchronous *anchor* circuit that runs occasional unmasked passes and uses their geometry to denoise the fast masked gradients without blocking them.

Asynchronous anchor circuit. A naive synchronous fix would periodically disable masking and run a full unmasked forward-backward pass through the main pipeline. At our target bandwidth this is prohibitively expensive: an unmasked pass sends roughly $1/(1-p)$ more activation data per boundary, and the entire fast circuit waits while it runs (Appendix H). Our alternative is to spin off a parallel anchor circuit that runs unmasked passes from a slightly stale weight snapshot, asynchronously, and feeds its gradients back to the fast circuit when they arrive. The fast circuit never waits. The cost is that anchor gradients are delayed (we measure $\Delta \approx 20$ –25 fast-circuit steps in our configuration), so they cannot be used as exact gradient updates, *i.e.*, they can only inform optimization *geometry*. This is the restriction that shapes the rest of the design.

A separate momentum buffer for unmasked gradients. For each targeted weight matrix W , we maintain an EMA of arriving anchor gradients,

$$M_t^{\text{anc}} = \beta_{\text{anc}} M_{t^-}^{\text{anc}} + (1 - \beta_{\text{anc}}) G_{\tau}^{\text{anc}}, \quad (1)$$

where t^- is the optimizer state immediately before the anchor gradient arrives. Two design choices here. First, we keep this momentum *separate* from AdamW’s first moment m_t rather than folding anchor gradients into it: m_t is updated on every fast step and is dominated by masked-step noise, so blending stale unmasked gradients into it would either dilute their signal or, with an aggressive blend, introduce slow oscillations. A separate buffer lets us treat anchor gradients on their own terms. Second, we use an EMA rather than the latest anchor gradient because individual anchor gradients are themselves noisy stochastic estimates. We choose β_{anc} large enough to average over several anchor cycles, since the gradient subspace drifts slowly during fine-tuning (Section E.3).

From momentum to a denoising operator: Given M_t^{anc} , we want to extract the directions in parameter space that are repeatedly reinforced by unmasked gradients, *i.e.*, the geometry the fast circuit’s noisy gradients should be pulled toward. Singular value decomposition is the natural choice: $M_t^{\text{anc}} = U_t S_t V_t^\top$ identifies left and right subspaces with strength quantified by the singular values s_i . Directions with large s_i are those the anchor circuit has consistently observed; directions with small s_i are either weakly supported or absent. We use SVD rather than, e.g., Hessian eigendecomposition (which we cannot afford at scale) or AdamW’s second moment v_t (which is dominated by masked-step variance and reflects *noise* structure, not signal structure).

Filtering $d_i = s_i/(s_i + \tau_p)$. Given the anchor basis (U_t, V_t) , we need to decide how to use it. Hard projection onto the top- r singular subspace is one option but requires choosing r , and discards information about how strongly each direction is supported. Instead, we apply a soft filter

$$d_i = \frac{s_i}{s_i + \tau_p}, \quad (2)$$

which has three useful properties. First, it is bounded between 0 and 1, so the filter never amplifies the gradient. Second, it is adaptive: directions with $s_i \gg \tau_p$ pass nearly unattenuated ($d_i \rightarrow 1$), while directions with $s_i \ll \tau_p$ are damped ($d_i \rightarrow 0$). Third, it requires only one hyperparameter τ_p , which sets the scale of "weakly supported," rather than a hard rank cutoff. The form $s/(s + \tau_p)$ is the standard Tikhonov-regularized soft threshold, familiar from inverse problems, and gives a Lipschitz filter that is well-behaved as the anchor momentum evolves.

Two-sided application. For a matrix gradient G_t^{mask} , we transform into the anchor basis, $X_t = U_t^\top G_t^{\text{mask}} V_t$, apply the filter to both index axes, $X_t^{\text{filt}} = \text{diag}(d) X_t \text{diag}(d)$, and reconstruct $G_t^{\text{filt}} = U_t X_t^{\text{filt}} V_t^\top$. Two-sided filtering reflects that for a matrix-valued gradient, both the row space (which input features are active) and the column space (which output features are active) carry information. The anchor circuit observes both; restricting to one side discards half the geometric information.

The final gradient passed to AdamW is

$$G_t^{\text{proj}} = \alpha G_t^{\text{mask}} + (1 - \alpha) G_t^{\text{filt}}. \quad (3)$$

We do not use the filtered gradient alone. The reason is that the anchor basis is an EMA of stale gradients, so G_t^{filt} inherits some staleness; the masked gradient is noisy but reflects the *current* weights. Blending preserves the current-iterate information while pulling toward the anchor's reliable subspace. The blend coefficient α trades these off: $\alpha = 1$ recovers masked-only training (no denoising), $\alpha = 0$ uses only the filtered gradient (maximum denoising, full inheritance of staleness), and intermediate values balance the two. Our convergence analysis (Appendix E) shows that this blend interpolates between the two regimes monotonically.

Putting these pieces together, the spectral correction is an online, adaptive preconditioner whose basis and weights are estimated by the anchor circuit and updated as new anchor gradients arrive. The masked gradient is never discarded. Instead, it is softly pulled toward directions the anchor circuit has consistently observed. Because the anchor circuit runs asynchronously, this denoising signal arrives at the cost of staleness, not throughput.

3.3 Data-Parallel Compression

PP compression alone does not suffice in a decentralized mesh because dense DP synchronization can dominate wall-clock time and erase the gains from masking. At Internet-grade bandwidth the disparity is severe. We therefore compose PP compression with DP compression. In our implementation we use Streaming DiLoCo [Douillard et al. \[2025\]](#) with PowerSGD [Vogels et al. \[2019\]](#): PowerSGD reduces bytes per synchronization via low-rank gradient compression, DiLoCo reduces synchronization frequency by performing local steps between outer updates, and Streaming DiLoCo overlaps the outer synchronization with local computation. Note that our PP-side mechanism (activation masking and anchor priors) is independent of this particular DP compressor and can be combined with better alternatives.

3.4 End-to-End Training Procedure

The fast circuit performs most optimizer steps while the The anchor circuit periodically pulls recent layer shards from the fast circuit, runs unmasked forward-backward passes, and returns the resulting gradients asynchronously. When an anchor gradient arrives, the fast circuit updates the anchor momentum M_t^{anc} and applies the spectral correction to subsequent masked gradients. The two-circuit design converts expensive unmasked computation into an asynchronous source of optimization geometry. The fast circuit retains the throughput benefits while the anchor circuit supplies delayed but reliable signals that stabilize the masked trajectory. Because the circuits are asynchronous, the system avoids the bandwidth spikes and pipeline stalls of synchronous unmasked steps. The overall system design is illustrated in Fig. 1

4 Convergence Analysis

We combine DP compression and PP activation masking. The DP component uses Streaming DiLoCo [\[Douillard et al., 2025\]](#) with PowerSGD [\[Vogels et al., 2019\]](#). We do not re-analyze this part:

Table 1: Per-domain task accuracy and global training efficiency for dense SFT and masked training variants. **Baseline**: dense PP, uncompressed DP. **+DP**: dense PP, compressed DP. **M90/M95**: 90%/95% activation masking; **+AP**: Anchor Priors. DP compression is applied to all the masked variants.

Domain	Dataset	Metric	Baseline	+DP	M90	M90+AP	M95	M95+AP
Medical	MedQA	Acc.	0.280	0.275	0.225	0.325	0.180	0.321
	MedMCQA	Acc.	0.330	0.325	0.275	0.335	0.220	0.320
Code	HumanEval	pass@1	0.177	0.171	0.115	0.215	0.072	0.207
	MBPP	pass@1	0.152	0.148	0.085	0.198	0.040	0.192
SQL	Spider	Exec.	0.423	0.412	0.305	0.418	0.180	0.402
Science	ARC-C	Acc.	0.378	0.374	0.328	0.380	0.295	0.355
	SciQ	Acc.	0.956	0.948	0.890	0.952	0.815	0.903
Commonsense	HellaSwag	Acc.	0.556	0.551	0.510	0.690	0.475	0.683
	WinoGrande	Acc.	0.548	0.541	0.512	0.628	0.488	0.625
Summ.	CNN/DM	R-1	0.178	0.176	0.140	0.205	0.108	0.195
	XSum	R-1	0.117	0.115	0.092	0.125	0.068	0.120
Math	GSM8K	EM	0.378	0.365	0.275	0.372	0.165	0.341
Doc. Parsing	QuALITY	Acc.	0.284	0.279	0.215	0.288	0.155	0.253
	NarrativeQA	Tok F1	0.699	0.685	0.485	0.682	0.295	0.557
TPS (tok/s)			1.1k	3.6k	29.3k	27.8k	48.9k	46.5k
Comm. (B/tok)			22.9k	6.9k	713	790	370	446

PowerSGD is a well-studied low-rank gradient compressor, while Streaming DiLoCo build on local-SGD/Federated-Averaging-style optimization. Both have been analyzed and validated extensively in prior work, and our method can in principle be paired with other DP compressors as well. We therefore focus on the masked activation training with asynchronous Anchor Priors.

Let $F(w) = \mathbb{E}_\xi[\ell(w; \xi)]$ denote the population training objective, F^* its lower bound, w_t the model weights at step t , η the learning rate, and L the smoothness constant of F . Let G bound the second moment of the update direction, and Δ the maximum staleness of anchor gradients in fast-circuit steps. The blend coefficient $\alpha \in [0, 1]$ controls how strongly the spectral correction is applied (Section 3).

Theorem 1 (Informal; full version in Appendix E) *Under standard smoothness and structural assumptions on the spectral filter, after T steps,*

$$\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} \|\nabla F(w_t)\|^2 \leq \frac{2(F(w_0) - F^*)}{\mu_\alpha \eta T} + \frac{\Sigma_{\alpha, \Delta}^2 + L\eta G^2}{\mu_\alpha},$$

where $\Sigma_{\alpha, \Delta}^2$ is a noise floor with three terms — unfiltered masked noise, filter-surviving noise (scaling as a contraction factor $\rho < 1$), and anchor staleness — and $\mu_\alpha = 1 - (1 - \alpha)^2 \kappa$ measures how much true gradient the filter preserves (with $\kappa \in [0, 1]$ the fraction of gradient energy the filter discards).

The bound has the standard nonconvex-SGD shape (a $1/T$ term plus a noise floor) and improves over masked-only training when the filter contracts noise ($\rho < 1$) more than it suppresses signal ($\kappa > 0$). Both quantities are small in the *fine-tuning* regime for two reasons: fine-tuning gradients live in a low-dimensional subspace [Aghajanyan et al., 2021, Hu et al., 2022], so the anchor basis preserves nearly all of $\nabla F(w_t)$ (κ small); and fine-tuning trajectories drift slowly [Neyshabur et al., 2020, Wortsman et al., 2022], so a delayed anchor basis remains close to the current one (staleness term controlled). Pretraining violates both, and the analysis correctly predicts no benefit there. The key takeaway is that the anchor gradients need not be exact synchronized updates, only good enough to estimate a low-rank subspace that is approximately preserved over the staleness window, the property the fine-tuning regime provides. The full proof is given in Appendix E.

5 Experiments

We evaluate the proposed system across diverse adaptation domains, including **general knowledge**, **medical**, **programming**, **math**, **science**, **commonsense reasoning**, **summarization**, and **document parsing**. We use PowerSGD ($\times 64$) and Streaming Diloco for DP. Our experiments span mesh sizes, model families, and model scales. We use A100 GPUs (40GB VRAM) for all our experiments.

5.1 Performance across different domain adaptation tasks

Table 1 summarizes the main domain-specific results. We use the publicly available Llama 3.2 1B checkpoint as the pretrained model: $\sim 1.23\text{B}$ parameters, LLaMA-style decoder-only architecture with 16 transformer layers, hidden dimension 2048, and 32 attention heads. All models are trained for 10 epochs on the corresponding training datasets with learning rate 2×10^{-5} , cosine decay, 2k context length, and batch size 32. The system runs on a 7×8 mesh for the masked (fast) circuit and a 1×8 mesh for the unmasked (clean) circuit, with all inter-node links throttled to 200 Mbps to emulate decentralized deployment. Datasets, evaluation metrics, and per-run configurations are in Appendix I.

As shown pure activation masking (M90, M95) is catastrophic on its own. Adding Anchor Priors reverses this collapse on every domain: M90+AP and M95+AP match the uncompressed Baseline within noise on most tasks and exceed it on several. **We hypothesize the superior performance on some datasets might be due to the regularization effect of masking leading to better generalization.** On the throughput, the system delivers substantially higher efficiency under bandwidth constraints: M95 reaches a $\sim 45\times$ TPS gain over the uncompressed baseline by stacking $20\times$ PP activation masking with $640\times$ DP gradient compression, while either compression alone yields only $1.4\times$ or $3.3\times$ because each merely shifts the bottleneck. We observe a practical delay of $\sim 20 - 25$ steps in the unmasked slow gradients.

5.2 Generalization across model variants

Table 3 reports results across Gemma, Qwen, Phi, LLaMA, and StableLM models, spanning 1B–8B parameters, 16–36 layers, and hidden dimensions from 1536 to 8192. We also evaluate the method across different DP $\times$ PP mesh sizes. In all settings, we use 95% activation masking with Anchor Priors (AP). Across these variations, the proposed method consistently achieves near-parity with the uncompressed baseline, showcasing its generalization across architectures and scale.

5.3 Ablations

We sweep the two knobs that most directly govern the fast/clean trade-off: the activation-mask ratio p and the Anchor-Prior staleness K (number of fast-circuit steps between consecutive uncompressed priors). Table 2 reports results. Two failure modes emerge. First, $p=99\%$ collapses at every K . At this masking ratio the per-step information budget on the PP edges is too small for the clean prior to repair. Second, $K=50$ collapses at every p . By the time a prior this stale arrives, the masked trajectory has drifted beyond its effective correction horizon (Appendix H). Inside the operating envelope ($p \in \{90, 95\}\%$, $K \in \{10, 20\}$) all four corners perform well. Finally, we ablate over τ and α .

Table 2: Ablations over mask ratio p , staleness K , spectral-filter threshold τ , and merge coefficient α . Default ($p=95\%$, $K=20$, $\tau=10^{-3}$, $\alpha=0.3$) in green; (b), (c) hold others at default. Red: $> 30\%$ relative drop.

(a) Mask ratio p vs. staleness K					
Dataset	Mask	$K=10$	$K=20$	$K=30$	$K=50$
MedQA	$p=90\%$	0.330	0.328	0.310	0.265
	$p=95\%$	0.328	0.321	0.295	0.250
	$p=99\%$	0.245	0.230	0.215	0.180
HumanEval	$p=90\%$	0.218	0.215	0.195	0.140
	$p=95\%$	0.215	0.207	0.175	0.090
	$p=99\%$	0.085	0.075	0.060	0.025
(b) Spectral threshold τ					
Dataset		10^{-5}	10^{-4}	10^{-3}	10^{-1}
MedQA		0.215	0.290	0.321	0.245
HumanEval		0.105	0.180	0.207	0.140
(c) Merge coefficient α					
Dataset		$\alpha=0.0$	$\alpha=0.3$	$\alpha=0.5$	$\alpha=1.0$
MedQA		0.260	0.321	0.310	0.180
HumanEval		0.105	0.207	0.195	0.072

Table 3: Code-domain generalization across model families and scales. Entries are HumanEval / MBPP pass@1. M95: 95% activation masking; AP: Anchor Priors. M95+AP recovers most of the dense performance across all scales tested.

Model	Params	Dense	M95	M95+AP
Llama-1B	1.0B	0.177 / 0.152	0.128 / 0.032	0.171 / 0.141
Qwen-1.5B	1.5B	0.213 / 0.188	0.158 / 0.069	0.206 / 0.177
Gemma-2B	2.0B	0.201 / 0.174	0.147 / 0.055	0.194 / 0.162
Phi-2	2.7B	0.238 / 0.205	0.176 / 0.083	0.228 / 0.193
Qwen-3B	3.0B	0.268 / 0.232	0.207 / 0.098	0.260 / 0.222
OLMo-7.5B	7.5B	0.335 / 0.294	0.267 / 0.153	0.324 / 0.280
Llama-8B	8.0B	0.384 / 0.338	0.309 / 0.184	0.372 / 0.323

Table 4: Comparison of activation-compression baselines. **Comp.**: forward/backward compression ratio. Red cells mark runs that failed to learn.

Method	Comp.	MedQA	HE	MBPP	GSM8K
Baseline (dense)	1×	0.28	0.18	0.15	0.38
Wang et al. [2022b]	8/2×	0.25	0.00	0.00	0.00
He et al. [2025]	8/2×	0.25	0.00	0.00	0.00
4-bit Quant	4×	0.25	0.13	0.11	0.18
Top- <i>k</i>	10×	0.23	0.12	0.09	0.16
Ramasinghe et al. [a]	10×	0.25	0.13	0.10	0.17
Bottleneck Proj.	20×	0.26	0.14	0.12	0.12
M95+AP (ours)	20×	0.32	0.21	0.19	0.34

Table 5: Robustness to post-hoc magnitude pruning. M95+AP retains a larger fraction of accuracy under pruning. “Avg. Δ ” is the mean relative drop across ARC-C, HellaSwag, MMLU.

Metric	Method	<i>s</i> =0	30%	50%	70%
ARC-C	Dense	0.33	0.26	0.25	0.25
	Ours	0.39	0.31	0.30	0.29
HellaSwag	Dense	0.57	0.41	0.27	0.25
	Ours	0.66	0.53	0.42	0.35
MMLU	Dense	0.35	0.27	0.23	0.23
	Ours	0.36	0.30	0.27	0.26
Avg. Δ	Dense	—	−25%	−37%	−40%
	Ours	—	−19%	−29%	−34%

To verify that the method does not depend on a particular mesh topology, we sweep the DP × PP grid from 4×4 up to 16×16 under the same M95+AP and 200 Mbps configuration as the main experiments. Table 6 reports MedQA, HumanEval, and GSM8K accuracy alongside aggregate throughput. As shown, the performance is stable across mesh sizes: all seven configurations land within $\pm 5\%$ of the 8×8 reference on every metric, including the most-stressed 16×16 case. The Anchor-Prior mechanism is therefore not tied to a specific topology. Second, throughput scales as expected: roughly linearly in DP−1 at fixed PP (more masked replicas trained in parallel), and inversely with T_{step} along the PP axis (deeper pipelines move proportionally more masked activations per step). The combination of stable accuracy and predictable throughput scaling supports the claim that the fast/clean-circuit decomposition is a property of the algorithm rather than an artifact of a specific configuration.

5.4 Comparison to Activation-Communication Baselines

Table 4 compares M95+AP against published activation- and gradient-compression methods across medical (MedQA, MedMCQA), code (HumanEval, MBPP), and math (GSM8K) domains. Two findings stand out. First, prior methods do not survive at our compression target. Aggressive 2-bit quantization (AC-SGD, TAH-Quant) collapses entirely on generative tasks such as HumanEval, MBPP, and GSM8K, and reduces medical accuracy to near-random. Sparsification and 4-bit quantization recover partially. In contrast, our method matches or exceeds the uncompressed Baseline at $20 \times / 20 \times$.

5.5 Robustness to Post-Hoc Pruning

A useful side-effect of training under aggressive activation masking is that the resulting checkpoint becomes more tolerant of post-hoc weight sparsification (Table 5). A plausible interpretation is that the masked-optimizer trajectory biases the trained weights toward solutions where information is distributed across many small-magnitude parameters, which is precisely what magnitude pruning rewards.

Table 6: Ablation over decentralized mesh sizes. Mesh size is DP×PP; one of the DP replicas is dedicated to the unmasked (clean) circuit and the remaining DP−1 replicas run the masked (fast) circuit. All runs use M95+AP at 200 Mbps with the same training configuration as the main experiments. **HE**: HumanEval pass@1. **TPS**: tokens per second.

Mesh	DP	PP	MedQA	HE	GSM8K	TPS
4 × 4	4	4	0.318	0.205	0.338	32,100
8 × 4	8	4	0.325	0.213	0.345	75,000
4 × 8	4	8	0.315	0.200	0.335	19,900
8 × 8	8	8	0.321	0.207	0.341	46,500
16 × 8	16	8	0.328	0.215	0.348	99,600
8 × 16	8	16	0.312	0.198	0.330	26,400
16 × 16	16	16	0.318	0.205	0.336	56,600

5.6 Reasoning and continual pretraining

Beyond per-domain fine-tuning, we test whether our method can sustain the longer training trajectories typical of continual pretraining and multi-stage SFT. We run a three-phase pipeline matching the SmolLM3 recipe on Llama-3.2-1B, : (1) reasoning-focused mid-training on Llama-Nemotron and

OpenThoughts traces, (2) a first SFT round on the smoltalk2 mixture, and (3) a second SFT round on the same mixture. The masked optimization preserves reasoning capability, showing that the spectral correction tracks the true gradient across substantially longer trajectories than the per-domain fine-tuning runs. Total training is $\sim 60\text{B}$ tokens. Full configurations in Appendix B. Learned reasoning traces are shown in Appendix C. Also, a natural concern with masked fine-tuning is whether it leaves the model in a state amenable to downstream RL. To this end, we perform an RL training on the checkpoints after the above steps and show that the compressed model does not affect the RL phase (Appendix D, Table 9).

6 Orthogonality to Parameter-Efficient Fine-Tuning

Parameter-efficient fine-tuning (PEFT) methods reduce the number of trainable parameters and optimizer states, but they do not reduce pipeline-parallel activation traffic. Even when only LoRA adapters are trained, each pipeline stage still consumes the full forward and backward traffic. Thus, PEFT improves memory and update efficiency, whereas our method targets the orthogonal bottleneck of PP communication. To verify this, we evaluate dense SFT, LoRA, and our method, and their combination on the code domain. LoRA reduces trainable parameters but leaves performance relatively unchanged. The results are shown in Table 8.

Table 7: Llama-3.2-1B trained through the full SmolLM3-style recipe (reasoning mid-training $\rightarrow$ SFT $\rightarrow$ second SFT round). Final model after all three phases; 60B total tokens. M95+AP is on par with dense quality at $\sim 43\times$ throughput.

Method	IFEval	GSM8K	TPS	Speedup
Base	0.13	0.02	—	—
Dense	0.46	0.23	1.1k	1 $\times$
M95+AP	0.41	0.21	46.5k	43 $\times$

Table 8: Orthogonality of M95+AP to parameter-efficient fine-tuning on Llama 3.2 1B. LoRA reduces trainable parameters but still requires dense activation and activation-gradient transfer across PP boundaries. M95+AP reduces PP communication independently and composes with LoRA, giving both axes of efficiency at once. **Comp.**: forward/backward activation compression ratio. **HE**: HumanEval pass@1.

Method	Trainable	PP fwd	PP bwd	Comp.	MedQA	MedMCQA	HE
Dense SFT	1.23B (100%)	Dense	Dense	1 $\times$ / 1 $\times$	0.280	0.330	0.177
LoRA	3.15M (0.26%)	Dense	Dense	1 $\times$ / 1 $\times$	0.265	0.318	0.168
M95+AP	1.23B (100%)	Sparse	Sparse	20 $\times$ / 20 $\times$	0.321	0.320	0.207
LoRA + M95+AP	3.15M (0.26%)	Sparse	Sparse	20 $\times$ / 20 $\times$	0.305	0.310	0.198

7 Conclusion

We presented a communication-efficient system for decentralized LLM adaptation over low-bandwidth GPU meshes. By revisiting activation masking as an in-graph PP compression mechanism and pairing it with asynchronous Anchor Priors, adaptive spectral correction, and DP compression, our method makes sparse activation training practical even at extreme compression rates. The convergence analysis identifies the regime under which this works, *i.e.*, low-dimensional, slowly-drifting gradient subspaces characteristic of fine-tuning, and explains why random PRF masking, rather than top- K , is the right primitive in this setting. Across domain adaptation and continual pretraining, the resulting system matches or exceeds dense uncompressed training while delivering large throughput gains, and the resulting models are more robust to post-hoc pruning. These results show that post-pretraining adaptation can tolerate sparser intermediate signals than previously assumed, suggesting a practical path toward large-model adaptation over decentralized collections of low-end GPUs.

Limitations

Our experiments and analysis target the fine-tuning regime; we do not claim the method extends to pretraining, where gradient subspaces are high-dimensional and shift substantially over training. Adaptation of dual EMA optimizers such as Adammix would be an interesting future direction (we have included a discussion in Appendix K). Further, our experiments are at the 1–8B parameter scale; scaling behavior at 70B+ remains an open question.

References

- Armen Aghajanyan, Sonal Gupta, and Luke Zettlemoyer. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP)*, pages 7319–7328. Association for Computational Linguistics, 2021.
- Thalaiyasingam Ajanthan, Sameera Ramasinghe, Gil Avraham, Hadi Mohaghegh Dolatabadi, Chamin P Hewa Koneputugodage, Violetta Shevchenko, Yan Zuo, and Alexander Long. Asyncmesh: Fully asynchronous optimization for data and pipeline parallelism. *arXiv preprint arXiv:2601.22442*, 2026.
- Alham Fikri Aji and Kenneth Heafield. Sparse communication for distributed gradient descent. In *EMNLP*, 2017.
- Dan Alistarh, Torsten Hoefler, Mikael Johansson, Nikola Konstantinov, Sarit Khirirat, and Cédric Renggli. The convergence of sparsified gradient methods. In *NeurIPS*, 2018.
- Debraj Basu, Deepesh Data, Can Karakus, and Suhas Diggavi. Qsparse-local-SGD: Distributed SGD with quantization, sparsification and local computations. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Jeremy Bernstein, Yu-Xiang Wang, Kamyar Azizzadenesheli, and Anima Anandkumar. signSGD: Compressed optimisation for non-convex problems. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *PMLR*, pages 559–568, 2018.
- Michael Diskin, Alexey Bukhtiyarov, Max Ryabinin, Lucile Saulnier, Quentin Lhoest, Anton Sinitsin, Dmitry Popov, Dmitry Pyrkun, Maxim Kashirin, Alexander Borzunov, Albert Villanova del Moral, Denis Mazur, Ilia Kobelev, Yacine Jernite, Thomas Wolf, and Gennady Pekhimenko. Distributed deep learning in open collaborations. In *Advances in Neural Information Processing Systems*, volume 34, 2021.
- Arthur Douillard, Qixuan Feng, Andrei A. Rusu, Rachita Chhaparia, Yani Donchev, Adhiguna Kuncoro, Marc’Aurelio Ranzato, Arthur Szlam, and Jiajun Shen. DiLoCo: Distributed low-communication training of language models. *arXiv preprint arXiv:2311.08105*, 2023a.
- Arthur Douillard, Qixuan Feng, Andrei A. Rusu, Rachita Chhaparia, Yani Donchev, Adhiguna Kuncoro, Marc’Aurelio Ranzato, Arthur Szlam, and Jiajun Shen. DiLoCo: Distributed low-communication training of language models. *arXiv preprint arXiv:2311.08105*, 2023b. First posted Nov 2023; latest revision Sep 2024.
- Arthur Douillard, Yanislav Donchev, Keith Rush, Satyen Kale, Zachary Charles, Zachary Garrett, Gabriel Teston, Dave Lacey, Ross McIlroy, Jiajun Shen, Alexandre Ramé, Arthur Szlam, Marc’Aurelio Ranzato, and Paul Barham. Streaming DiLoCo with overlapping communication: Towards a distributed free lunch. *arXiv preprint arXiv:2501.18512*, 2025.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 7 2024. URL <https://zenodo.org/records/12608602>.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Frithjof Gressmann, Zach Eaton-Rosen, and Carlo Luschi. Improving neural network training in low dimensional random bases. In *Advances in Neural Information Processing Systems*, volume 33, pages 12140–12150, 2020.
- Guy Gur-Ari, Daniel A. Roberts, and Ethan Dyer. Gradient descent happens in a tiny subspace. *arXiv preprint arXiv:1812.04754*, 2018.

Guangxin He, Yuan Cao, Yutong He, Tianyi Bai, Kun Yuan, and Binhang Yuan. TAH-Quant: Effective activation quantization in pipeline parallelism over slow network. *arXiv preprint arXiv:2506.01352*, 2025.

Neil Houlsby, Andrei Giurgiu, Stanisław Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *PMLR*, pages 2790–2799, 2019.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Sami Jaghouar, Jack Min Ong, and Johannes Hagemann. OpenDiLoCo: An open-source framework for globally distributed low-communication training. *arXiv preprint arXiv:2407.07852*, 2024a.

Sami Jaghouar, Jack Min Ong, and Johannes Hagemann. OpenDiLoCo: An open-source framework for globally distributed low-communication training. *arXiv preprint arXiv:2407.07852*, 2024b.

Sai Praneeth Karimireddy, Quentin Rebjock, Sebastian U. Stich, and Martin Jaggi. Error feedback fixes signsgd and other gradient compression schemes. In *ICML*, 2019.

Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3045–3059. Association for Computational Linguistics, 2021.

Chunyuan Li, Heerad Farkhor, Rosanne Liu, and Jason Yosinski. Measuring the intrinsic dimension of objective landscapes. In *International Conference on Learning Representations (ICLR)*, 2018.

Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP)*, pages 4582–4597. Association for Computational Linguistics, 2021.

Vladislav Lialin, Sherin Muckatira, Namrata Shivagunde, and Anna Rumshisky. ReLoRA: High-rank training through low-rank updates. *arXiv preprint arXiv:2307.05695*, 2023.

Yujun Lin, Song Han, Huizi Mao, Yu Wang, and William J. Dally. Deep gradient compression: Reducing the communication bandwidth for distributed training. In *ICLR*, 2018a.

Yujun Lin, Song Han, Huizi Mao, Yu Wang, and William J. Dally. Deep gradient compression: Reducing the communication bandwidth for distributed training. In *International Conference on Learning Representations (ICLR)*, 2018b.

Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prithvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.

Behnam Neyshabur, Hanie Sedghi, and Chiyuan Zhang. What is being transferred in transfer learning? In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Matteo Pagliardini, Pierre Ablin, and David Grangier. The AdEMAMix optimizer: Better, faster, older. *arXiv preprint arXiv:2409.03137*, 2024.

Vardan Papyan. Traces of class/cross-class structure pervade deep learning spectra. *Journal of Machine Learning Research*, 21(252):1–64, 2020.

Pitch Patarasuk and Xin Yuan. Bandwidth optimal all-reduce algorithms for clusters of workstations. *Journal of Parallel and Distributed Computing*, 69(2):117–124, 2009. doi: 10.1016/j.jpdc.2008.09.002.

Bowen Peng, Jeffrey Quesnelle, and Diederik P. Kingma. DeMo: Decoupled momentum optimization. *arXiv preprint arXiv:2411.19870*, 2024.

494 Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. Adapter-
495 Fusion: Non-destructive task composition for transfer learning. In *Proceedings of the 16th*
496 *Conference of the European Chapter of the Association for Computational Linguistics (EACL)*,
497 pages 487–503. Association for Computational Linguistics, 2021.

498 Vinay V. Ramasesh, Ethan Dyer, and Maithra Raghu. Anatomy of catastrophic forgetting: Hidden
499 representations and task semantics. In *International Conference on Learning Representations*
500 *(ICLR)*, 2021.

501 Sameera Ramasinghe, Thalaiyasingam Ajanthan, Gil Avraham, Yan Zuo, and Alexander Long.
502 Beyond top-k: Structured sparsification for compression in pipeline parallel. In *ICLR 2025*
503 *Workshop on Modularity for Collaborative, Decentralized, and Continual Deep Learning*, a.

504 Sameera Ramasinghe, Thalaiyasingam Ajanthan, Hadi Mohaghegh Dolatabadi, Gil Avraham, Violetta
505 Shevchenko, Yan Zuo, Chamin P Hewa Koneputugodage, and Alexander Long. Mixtures of
506 subspaces for bandwidth efficient context parallel training. In *The Thirty-ninth Annual Conference*
507 *on Neural Information Processing Systems*, b.

508 Sameera Ramasinghe, Thalaiyasingam Ajanthan, Gil Avraham, Yan Zuo, and Alexander Long.
509 Subspace networks: Scaling decentralized training with communication-efficient model parallelism.
510 In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

511 Mikhail I Rudakov, Aleksandr Nikolaevich Beznosikov, Ya A Kholodov, and Alexan-
512 der Vladimirovich Gasnikov. Activations and gradients compression for model-parallel training.
513 In *Doklady Mathematics*, volume 108, pages S272–S281. Springer, 2023.

514 Max Ryabinin, Tim Dettmers, Michael Diskin, and Alexander Borzunov. SWARM parallelism:
515 Training large models can be surprisingly communication-efficient. In *Proceedings of the 40th*
516 *International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 29416–
517 29440, 2023.

518 Lorenzo Sani, Alex Jacob, Zeyu Cao, Royson Lee, Bill Marino, Yan Gao, Dongqi Cai, Zexi Li, Wanru
519 Zhao, Xinchu Qiu, and Nicholas D. Lane. Photon: Federated LLM pre-training. *arXiv preprint*
520 *arXiv:2411.02908*, 2024.

521 Sebastian U. Stich. Local SGD converges fast and communicates little. In *International Conference*
522 *on Learning Representations (ICLR)*, 2019.

523 Sebastian U. Stich, Jean-Baptiste Cordonnier, and Martin Jaggi. Sparsified sgd with memory. In
524 *NeurIPS*, 2018.

525 Thijs Vogels, Sai Praneeth Karimireddy, and Martin Jaggi. PowerSGD: Practical low-rank gradient
526 compression for distributed optimization. In *Advances in Neural Information Processing Systems*,
527 volume 32, 2019.

528 Hongyi Wang, Scott Sievert, Shengchao Liu, Zachary Charles, Dimitris Papailiopoulos, and Stephen
529 Wright. ATOMO: Communication-efficient learning via atomic sparsification. In *Advances in*
530 *Neural Information Processing Systems*, volume 31, 2018.

531 Hongyi Wang, Saurabh Agarwal, and Dimitris Papailiopoulos. Pufferfish: Communication-efficient
532 models at no extra cost. *Proceedings of Machine Learning and Systems*, 3:365–386, 2021.

533 Jue Wang, Binhang Yuan, Luka Rimanic, Yongjun He, Tri Dao, Beidi Chen, Christopher Ré, and
534 Ce Zhang. Fine-tuning language models over slow networks using activation quantization with
535 guarantees. In *Advances in Neural Information Processing Systems*, volume 35, 2022a.

536 Jue Wang, Binhang Yuan, Luka Rimanic, Yongjun He, Tri Dao, Beidi Chen, Christopher Ré, and
537 Ce Zhang. Fine-tuning language models over slow networks using activation quantization with
538 guarantees. *Advances in Neural Information Processing Systems*, 35:19215–19230, 2022b.

539 Per-Åke Wedin. Perturbation bounds in connection with singular value decomposition. *BIT Numerical*
540 *Mathematics*, 12(1):99–111, 1972. doi: 10.1007/BF01932678.

541 Mitchell Wortsman, Gabriel Ilharco, Jong Wook Kim, Mike Li, Simon Kornblith, Rebecca Roelofs,
542 Raphael Gontijo Lopes, Hannaneh Hajishirzi, Ali Farhadi, Hongseok Namkoong, and Ludwig
543 Schmidt. Robust fine-tuning of zero-shot models. In *Proceedings of the IEEE/CVF Conference on*
544 *Computer Vision and Pattern Recognition (CVPR)*, pages 7959–7971, 2022.

545 Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian
546 Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng,
547 Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie
548 Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-
549 Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An
550 open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

551 Yi Yu, Tengyao Wang, and Richard J. Samworth. A useful variant of the Davis–Kahan theorem for
552 statisticians. *Biometrika*, 102(2):315–323, 2015. doi: 10.1093/biomet/asv008.

553 Binhang Yuan, Yongjun He, Jared Davis, Tianyi Zhang, Tri Dao, Beidi Chen, Percy Liang, Christopher
554 Ré, and Ce Zhang. Decentralized training of foundation models in heterogeneous environments.
555 In *Advances in Neural Information Processing Systems*, volume 35, pages 25464–25477, 2022.

556 Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong
557 Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection. In *International*
558 *Conference on Machine Learning (ICML)*, 2024a.

559 Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong
560 Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection. In *International*
561 *Conference on Machine Learning (ICML)*, 2024b.

562 Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong
563 Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization.
564 *arXiv preprint arXiv:2507.18071*, 2025.

Require: Pretrained weights w_0 , mask probability p , anchor weight α , anchor EMA β_{anc} , damping

```

 $\tau_p$ 
1: Split mesh into fast masked circuit and asynchronous anchor circuit
2: Initialize  $w^{\text{fast}}, w^{\text{anc}} \leftarrow w_0, M^{\text{anc}} \leftarrow 0$ 
3: for step  $t = 1, \dots, T$  do
4:   Forward through PP with deterministic in-graph masks  $M_\ell = g(\ell, t, B, S, H)$ :  $\tilde{h}_\ell = h_\ell \odot M_\ell$ 
5:   Backward inherits the same sparsity:  $\partial \mathcal{L} / \partial h_\ell = (\partial \mathcal{L} / \partial \tilde{h}_\ell) \odot M_\ell$ 
6:   Compute  $G_t^{\text{mask}}$ ; synchronize DP replicas with compressed all-reduce
7:   if anchor gradient  $G_t^{\text{anc}}$  has arrived then
8:      $M_t^{\text{anc}} \leftarrow \beta_{\text{anc}} M_{t-1}^{\text{anc}} + (1 - \beta_{\text{anc}}) G_t^{\text{anc}}$ 
9:   end if
10:  for each targeted matrix  $W$  do
11:     $M_t^{\text{anc}} = U_t S_t V_t^\top$ ;  $X_t = U_t^\top G_t^{\text{mask}} V_t$ 
12:     $d_i = s_i / (s_i + \tau_p)$ ;  $G_t^{\text{filt}} = U_t \text{diag}(d) X_t \text{diag}(d) V_t^\top$ 
13:     $G_t^{\text{proj}} = \alpha G_t^{\text{mask}} + (1 - \alpha) G_t^{\text{filt}}$ 
14:  end for
15:  Update  $w^{\text{fast}}$  via AdamW with  $G_t^{\text{proj}}$ 
16:  Anchor circuit asynchronously refreshes weights, runs unmasked forward-backward, returns  $G_t^{\text{anc}}$ 
17: end for

```

565 A Training Algorithm

566 The full training algorithm for our method is shown in Algorithm A.

567 B Reasoning and Continual-Pretraining Recipe

568 This section documents the three-phase training recipe used in Section 5.6. We follow the SmolLM3
569 methodology, but start from a smaller base model (Llama-3.2-1B) to make the experiment tractable
570 at our compute budget.

571 B.1 Base model

572 We use Llama-3.2-1B as the starting point, with its tokenizer extended by SmolLM3 ChatML
573 special tokens (`<|im_start|>`, `<|im_end|>`). The model is loaded in BF16 and trained with
574 FlashAttention-2 throughout.

575 B.2 Phase 1: Reasoning mid-training

576 **Data.** HuggingFaceTB/smoltalk2, config Mid:

- 577 • Llama_Nemotron_Post_Training_Dataset_reasoning_r1 (weight 1.0).
- 578 • OpenThoughts3_1.2M (weight 1.0).

579 **Configuration.** Plain ChatML chat template (no system metadata, no thinking-mode header).
580 AdamW with peak learning rate 2×10^{-5} , cosine schedule with min-LR ratio 0.1, warmup ratio 0.03,
581 gradient norm clipped at 0.2, sequence length 32,768, packing enabled, 2 training epochs, gradient
582 accumulation 2 on $8 \times$ A100, Liger kernel enabled. No assistant-only loss masking in this phase; loss
583 is computed over all tokens.

584 B.3 Phase 2: First SFT round

585 **Data.** The smoltalk2 SFT mixture, comprising 25 splits with weights ranging from 0.02 to 1.0.
586 Splits include conversational data (everyday conversations, system chats), instruction-following
587 (Tulu-3 personas, OpenHermes-2.5), tool-use (Hermes function calling, xLAM, smolagents traces),
588 reasoning traces (multi-turn-reasoning, Qwen3-32B-distilled think traces, s1k, OpenThoughts3 think

variant), multilingual data (smoltalk multilingual, Aya), long-context (LongAlign 64k), and structured data (table-GPT). The full per-split weights are listed in the supplementary YAML.

Configuration. SmolLM3 ChatML template with thinking-mode metadata (`<|im_start|>system` n## Metadata n). Loss masked over assistant tokens only (`assistant_only_loss: true`). Same optimizer as Phase 1 except: sequence length 65,536 (long-context splits), 5 training epochs, BFD packing strategy, save every 0.2 epochs.

B.4 Phase 3: Second SFT round

Data. Identical to Phase 2 (same smoltalk2 SFT mixture and split weights).

Configuration. Identical to Phase 2 except: starts from the Phase 2 checkpoint, 5 training epochs. The purpose of the second round is to extend total training while keeping the data distribution fixed — this tests whether masked optimization remains stable as the model approaches the data’s effective ceiling.

Some example reasoning traces of the compressed model are shown below.

C Reasoning Trace Examples

This appendix shows representative reasoning traces from the gshasiri/SmolLM3-Mid checkpoint after Phase 1 (reasoning mid-training) of the recipe in Section 5.6. Traces are produced with greedy decoding, `enable_thinking=True`, and `max_new_tokens=600`. Each example shows the full `<think>...</think>` block followed by the final answer.

Five examples below are the cleanly-correct answers from a 15-question simple-query batch. The remaining ten either produced incorrect answers or degenerated into repetition; this is expected because the SmolLM3-Mid checkpoint at this stage has chat-template scaffolding for thinking mode but has not yet undergone instruction/reasoning fine-tuning. The traces are intended to illustrate the *format* of the thinking output produced by the model after masked mid-training, not to demonstrate accuracy claims; per-task accuracy numbers are reported in Table 7.

Example 1: Square Perimeter

Question: A square has sides of 5 cm. What is its perimeter?

Thinking: Okay, so I need to find the perimeter of a square with a side length of 5 cm. The perimeter of a square is calculated by adding up all the sides. The formula for the perimeter of a square is 4 times the length of one of its sides. So, if the side is 5 cm, then the perimeter would be $4 \times 5 = 20$ cm. The side length is given, so the perimeter is straightforward.

Final Answer: The perimeter of the square is 20 cm.

Answer: To find the perimeter of a square with a side length of 5 cm, we use the formula:

$$\text{Perimeter} = 4 \times \text{side length} = 4 \times 5 = 20 \text{ cm}$$

20

Example 2: Half of a Number

Question: What is half of 30?

Thinking: Okay, I need to find half of 30. Half of a number means dividing the number by 2. So if I divide 30 by 2, that should give me 15. Let me check: $30/2 = 15$.

Final Answer: Half of 30 is 15.

Answer: To find half of 30, we divide 30 by 2.

15

617

Example 3: Rectangle Area

Question: A rectangle is 4 meters wide and 6 meters long. What is its area in square meters?

Thinking: The area of a rectangle is calculated by multiplying the length and the width. So, if the width is 4 meters and the length is 6 meters, then the area should be 4×6 . Let me write that down: $4 \times 6 = 24$ square meters.

Answer: The area of a rectangle with width w and length l is $A = lw$. For a rectangle 4 m wide and 6 m long:

$$A = 4 \times 6 = 24 \text{ m}^2$$

24

618

Example 4: Time-Unit Conversion

Question: How many minutes are there in one hour and a half?

Thinking: First, I should convert the given time into hours. “One hour and a half” — one hour is 60 minutes, and a half is 30 minutes. So 1 hour and a half is 1.5 hours.

Now, convert that to minutes. The formula for converting time to minutes is minutes = hours $\times$ 60. So $1.5 \times 60 = 90$ minutes.

Double-check: 1 hour is 60 minutes, 1.5 hours is $1.5 \times 60 = 90$ minutes. Correct.

Answer: The time in hours is 1.5 hours. Converting to minutes: $1.5 \times 60 = 90$ minutes.

90

619

Example 5: Triangle Perimeter (with Triangle-Inequality Check)

Question: A triangle has sides of length 3, 4, and 5 cm. What is its perimeter?

Thinking: I remember that the perimeter of a triangle is the sum of all three sides. First, let me verify it's a valid triangle. The triangle inequality says that the sum of any two sides must be greater than the third.

- $3 + 5 > 4$? $8 > 4$, yes.
- $4 + 5 > 3$? $9 > 3$, yes.
- $3 + 4 > 5$? $7 > 5$, yes.

All three inequalities hold, so the triangle is valid. Now to find the perimeter, add all three sides: $3 + 4 + 5 = 12$ cm.

Answer: The triangle has sides 3, 4, 5 cm. The perimeter is the sum of the sides: $3 + 4 + 5 = 12$ cm.

12

620

Example 6: Sum of First n Integers (Uses Formula)

Question: What is the sum of $1 + 2 + 3 + 4 + 5$?

621

Thinking: There's a formula for the sum of the first n natural numbers: $n(n+1)/2$. For $n = 1$, that gives $1 \cdot 2/2 = 1$, which matches. For $n = 2$, $2 \cdot 3/2 = 3$, which is correct. Plug in $n = 5$: $5 \cdot 6/2 = 15$. And directly: $1 + 2 + 3 + 4 + 5 = 15$. Both agree.

Answer: The sum of the first five natural numbers is

$$\frac{n(n+1)}{2} = \frac{5 \times 6}{2} = 15$$

15

622

Example 7: Boiling Point

Question: At what temperature does water boil at sea level, in degrees Celsius?

Thinking: The boiling point of water at sea level is 100 degrees Celsius. At sea level the atmospheric pressure is 1 atm, and at this pressure water boils at 100 °C.

Final Answer: The boiling point of water at sea level is 100 °C.

Answer: The boiling point of water at sea level is 100 degrees Celsius.

100 °C

623

Example 8: Chemical Formula

Question: What is the chemical formula for water?

Thinking: Water is a compound made up of hydrogen and oxygen atoms. The chemical formula for water is H₂O — one oxygen atom bonded to two hydrogen atoms.

Answer: The chemical formula for water is **H₂O**: two hydrogen atoms bonded to one oxygen atom.

H₂O

624

Example 9: Two-Step Word Problem (Multiplication)

Question: Sara has 3 boxes, each containing 4 apples. How many apples does Sara have in total?

Thinking: When you have multiple boxes, each with the same number of apples, the total is the number of boxes times the number per box. If there are n boxes each with k apples, the total is $n \times k$.

For Sara: $n = 3$, $k = 4$. Total = $3 \times 4 = 12$.

Sanity-check with a smaller case: 2 boxes of 2 apples each gives $2 \times 2 = 4$. That's right.

Answer: Sara has $3 \times 4 = 12$ apples in total.

12

625

626 C.1 Chat template

627 The SmolLM3 ChatML template used in Phases 2 and 3 injects per-turn system metadata (knowledge
628 cutoff date, current date, reasoning mode /think or /no_think) and supports XML-style tool
629 definitions. Reasoning content is wrapped in <think>...</think> tags. The template handles
630 three reasoning regimes:

Table 9: GSM8K performance before and after 2 epochs of GSPO on Big-Math, starting from either the dense or M95+AP final checkpoint. RL improvements are comparable from both starting points, indicating that masked fine-tuning leaves the model RL-amenable.

SFT starting point	Pre-RL	Post-RL	Δ
Dense (3-phase)	0.23	0.41	+0.18
M95+AP (3-phase)	0.21	0.40	+0.19

- *System override*: a system message containing `/system_override` replaces all metadata with user-supplied instructions verbatim.
- *Think mode*: assistant turns may include reasoning inside `<think>...</think>` tags. Used for splits ending in `_think`.
- *No-think mode*: assistant turns are emitted with an empty `<think></think>` block, signaling the model to skip reasoning. Used for splits ending in `_no_think`.

Phase 1 uses a simpler ChatML template without the metadata header, since the reasoning-trace data already encodes thinking-mode structure inline.

C.2 Activation masking and Anchor-Prior settings

Identical across all three phases and matching the per-domain experiments (Appendix I): activation mask probability 0.95, masking applied at layers $\{1, 3, 5, 7, 9, 11, 13\}$. Anchor-Prior gradients are injected with the spectral filter described in Section 3. The results are reported in Table 7.

C.3 Evaluation

IFEval and GSM8K are evaluated via the `lm-evaluation-harness` [Gao et al., 2024] on the final checkpoint after Phase 3. Token counts are measured as the cumulative training tokens across all three phases (sequence length $\times$ optimizer steps $\times$ effective batch).

D GSPO Reinforcement Learning Recipe

D.1 Reinforcement learning after masked SFT

A natural concern with masked fine-tuning is whether it leaves the model in a state amenable to downstream RL. Aggressive activation compression could in principle damage internal representations in ways that only manifest when the policy must be optimized against a sparse reward. For instance, by collapsing the entropy of intermediate features or by misaligning the model’s confidence estimates. We test this directly: starting from the M95+AP-trained final checkpoint of Section 5.6, we apply GSPO [Zheng et al., 2025] with a verifiable math reward on the Big-Math dataset and compare against the same RL run starting from the dense final checkpoint (Table 9).

The two starting points show essentially identical RL gains, and the post-RL gap matches the pre-RL gap. Masked SFT does not impair RL-amenable: the policy distribution remains entropic enough to explore, the reward landscape remains learnable, and the gradient signal from verifiable rewards translates into the same magnitude of capability gain whether the SFT trajectory was dense or communication-compressed.

Below we detail the GSPO [Zheng et al., 2025] configuration used in the RL phase. The setup follows VeRL’s reference GSPO recipe with adjustments for our 1B-parameter model and our context-length budget.

D.2 Pipeline

The full pipeline from base model to post-RL checkpoint is:

$$\text{Llama-3.2-1B} \xrightarrow{\text{Phase 1-3 SFT}} \text{SFT model} \xrightarrow{\text{DPO}} \text{DPO model} \xrightarrow{\text{GSPO}} \text{Final policy}$$

666 We run this pipeline twice: once with dense SFT throughout, and once with M95+AP SFT throughout
667 (Phases 1–3 of Appendix B). The DPO and GSPO phases are identical in both runs (same data,
668 hyperparameters, and no masking), so any difference in the post-RL policy is attributable to the
669 masked SFT trajectory.

670 D.3 Reward model

671 We use a verifiable math reward derived from the Big-Math dataset: each problem has a known final
672 answer, and the reward is a binary indicator of whether the model’s extracted answer matches. We
673 use the DAPO [Yu et al., 2025] reward manager, which adds a length-penalty buffer on top of the
674 correctness signal: responses that exceed an `overlong_buffer_len` threshold within the maximum
675 response length are linearly penalized down to a maximum penalty factor. This discourages reward
676 hacking via verbose answers.

677 Settings.

- 678 • `max_response_length`: 7,680 tokens (out of 8,192-token total context, leaving 512 for
679 the prompt).
- 680 • `overlong_buffer_len`: 3,584. Responses up to $7,680 - 3,584 = 4,096$ tokens incur no
681 length penalty; beyond that, reward is linearly attenuated.
- 682 • `overlong_penalty_factor`: 1.0.

683 D.4 GSPO algorithm settings

684 **Policy loss.** `loss_mode = "gspe"`: GSPO’s sequence-level importance ratio, with loss aggrega-
685 tion `seq-mean-token-mean` (mean over tokens within a response, then mean over responses).

686 **Advantage estimator.** `adv_estimator = "grpo"`: group-relative advantage with no critic. Ad-
687 vantages are computed as the standardized rewards within each group of N rollouts sharing a prompt.

688 **Group size.** $N = 16$ responses per prompt.

689 **Clip ratios.** Lower clip 3×10^{-4} , upper clip 4×10^{-4} . These are substantially smaller than standard
690 PPO clip ranges (0.2) because GSPO’s importance ratio is at the sequence level (the product of
691 per-token ratios over a long response). A small per-sequence ratio window corresponds to a much
692 larger effective per-token tolerance.

693 **KL settings.** We disable both KL-in-reward and KL-as-loss (`use_kl_in_reward = false`,
694 `use_kl_loss = false`). This is the standard RLVR setup where the only optimization signal
695 is the verifiable reward.

696 D.5 Optimization

- 697 • Optimizer: AdamW, learning rate 1×10^{-6} , weight decay 0.1, gradient clipping 1.0, warmup
698 ratio 0.05. Entropy regularization disabled (`entropy_coeff = 0`).
- 699 • Train batch size: 32 prompts per PPO iteration. With $N = 16$ rollouts each, this gives 512
700 trajectories per update.
- 701 • PPO mini-batch: 16 prompts (32 mini-batches per outer iteration).
- 702 • Per-GPU micro-batch: 4 prompts.
- 703 • Dynamic batching enabled (`use_dynamic_bsz = true`), capping per-GPU tokens at
704 8,192.

705 D.6 Rollout (vLLM)

- 706 • Backend: vLLM in asynchronous mode (rollouts overlap with actor training).
- 707 • GPU memory utilization for vLLM: 0.7.

- Tensor parallel: 1 (sufficient for a 1B model).
- Sampling for training rollouts: temperature 1.0, top- p 1.0, top- k disabled — maximally diverse exploration.
- Sampling for validation: temperature 1.0, top- p 0.7, one sample per prompt — a slightly more conservative distribution for cleaner eval signal.

D.7 Training schedule

- 2 epochs over Big-Math train split, with 25-step validation cadence and 100-step checkpoint cadence.
- No initial validation pass before training (`val_before_train = false`).
- DeepSpeed ZeRO-3 disabled in favor of FSDP; no parameter or optimizer offload (the 1B model fits comfortably on 40 GB GPUs).
- Sequence parallelism disabled (`sp_size = 1`).
- Gradient checkpointing enabled.

D.8 Evaluation

GSM8K is evaluated via the `lm-evaluation-harness` [Gao et al., 2024] on the final checkpoint after 2 epochs. We report exact-match accuracy at temperature 0 greedy decoding, matching the standard GSM8K evaluation protocol. Reported numbers are the best across all checkpoints of the corresponding run, with checkpoints saved every 100 training steps.

E Convergence Analysis with Spectral Anchor Correction

We analyze a simplified version of our optimizer that captures the role of asynchronous anchor priors and spectral gradient correction. The analysis formalizes a single tradeoff: masked gradients are cheap but noisy, while unmasked anchor gradients are clean but delayed. The spectral filter built from the anchor momentum is the mechanism that trades one for the other. We work in the *fine-tuning* regime, where the optimization trajectory is short and the gradient subspace drifts slowly — the structural property that makes a delayed anchor basis useful.

E.1 Setup

Let $F(w) = \mathbb{E}_{\xi}[\ell(w; \xi)]$ denote the population objective. At iteration t , the fast circuit computes a masked gradient g_t^{mask} , and the anchor circuit asynchronously produces a delayed unmasked gradient g_{τ}^{anc} from a stale snapshot w_{τ} with $t - \tau \leq \Delta$. We use Δ for the maximum staleness in fast-circuit steps to avoid clashing with the DP replica count D in Section 3.3.

For each matrix-valued parameter W , we maintain an EMA anchor momentum

$$M_t^{\text{anc}} = \beta_{\text{anc}} M_{t-}^{\text{anc}} + (1 - \beta_{\text{anc}}) g_{\tau}^{\text{anc}},$$

updated each time a new anchor gradient arrives. Its SVD $M_t^{\text{anc}} = U_t S_t V_t^{\top}$ defines the spectral filter

$$d_i = \frac{s_i}{s_i + \lambda_p}, \quad D_t = \text{diag}(d_1, \dots, d_r), \quad \mathcal{P}_t(G) = U_t D_t U_t^{\top} G V_t D_t V_t^{\top},$$

where $\lambda_p > 0$ is the spectral damping constant (distinct from the staleness index τ). The update direction is $u_t = \alpha g_t^{\text{mask}} + (1 - \alpha) \mathcal{P}_t(g_t^{\text{mask}})$, yielding $w_{t+1} = w_t - \eta u_t$.

E.2 Assumptions

Assumption 1 (Smoothness) F is L -smooth and bounded below by F^* .

Assumption 2 (Masked-gradient noise) $g_t^{\text{mask}} = \nabla F(w_t) + \varepsilon_t$ where ε_t is generated by a fresh PRF mask draw at step t , independent of all prior history. Conditional on that history, $\mathbb{E} \varepsilon_t = 0$ and $\mathbb{E} \|\varepsilon_t\|^2 \leq \sigma_m^2$.

747 **Assumption 3 (Delayed anchor gradients)** $g_\tau^{\text{anc}} = \nabla F(w_\tau) + \zeta_\tau$ with $t - \tau \leq \Delta$, $\mathbb{E} \zeta_\tau = 0$, and
748 $\mathbb{E} \|\zeta_\tau\|^2 \leq \sigma_a^2$.

749 **Assumption 4 (Bounded update)** $\mathbb{E} \|u_t\|^2 \leq G^2$ for all t .

750 The next two assumptions describe what the spectral filter does to the signal and to the noise. They
751 are the core structural conditions; we justify them in Section E.3 and quantify them in the supporting
752 lemmas.

753 **Assumption 5 (Signal preservation)** There exist $\kappa \in [0, 1)$ and $\delta_\Delta^{\text{stale}} \geq 0$ such that

$$\mathbb{E} \|(I - \mathcal{P}_t) \nabla F(w_t)\|^2 \leq \kappa \mathbb{E} \|\nabla F(w_t)\|^2 + \delta_\Delta^{\text{stale}}.$$

754 **Assumption 6 (Noise contraction)** There exists $\rho \in [0, 1]$ such that $\mathbb{E} \|\mathcal{P}_t \varepsilon_t\|^2 \leq \rho \sigma_m^2$.

755 E.3 Why these assumptions hold for fine-tuning

756 Both assumptions exploit two structural properties of fine-tuning that are well-documented in the
757 literature: (i) fine-tuning gradients have low effective rank, and (ii) fine-tuning trajectories drift slowly
758 in parameter space.

759 **Low-rank gradient structure.** The intrinsic dimension of fine-tuning objectives is small [Li et al.,
760 2018, Aghajanyan et al., 2021], the gradient and Hessian spectra during training are dominated by
761 a handful of outlier directions [Gur-Ari et al., 2018, Pappan, 2020, Gressmann et al., 2020], and
762 fine-tuning updates $\Delta W = W_t - W_0$ are well approximated by low-rank matrices [Hu et al., 2022,
763 Zhao et al., 2024b, Lialin et al., 2023]. Together these say $\nabla F(w_t)$ lies near a low-rank subspace, so
764 the principal directions of M_t^{anc} — itself an EMA of recent unmasked gradients — contain most of
765 $\nabla F(w_t)$ ’s energy. This makes κ small (Assumption 5).

766 **Slow weight drift.** Successful fine-tuning trajectories remain close to the pretrained initialization,
767 with feature representations changing modestly [Neyshabur et al., 2020, Ramasesh et al., 2021], and
768 fine-tuned checkpoints are close enough in parameter space to admit linear interpolation [Wortsman
769 et al., 2022]. Slow drift bounds the rotation of the gradient subspace between w_τ and w_t , which
770 controls $\delta_\Delta^{\text{stale}}$ via a Davis–Kahan argument (Lemma 3).

771 **Isotropic mask noise.** The mask $m_{t,j}$ is drawn from a PRF whose seed is independent of model
772 state, so ε_t is approximately isotropic in parameter space. A low-rank filter applied to isotropic noise
773 contracts it strongly: $\rho = O(r^2/(mn))$ in matrix dimensions $m \times n$ with effective rank r (Lemma 2).
774 This makes ρ small (Assumption 6).

775 These properties fail in pretraining: the gradient subspace is high-dimensional and shifts substantially
776 over a long trajectory. The analysis below predicts no benefit there, consistent with our empirical
777 scoping of the method to fine-tuning workloads.

778 E.4 Supporting lemmas

779 **Lemma 1 (Non-expansiveness of $\mathcal{P}_t$)** For any matrix G , $\|\mathcal{P}_t(G)\|_F \leq \|G\|_F$.

780 **Proof 1** Let $P_U = U_t D_t U_t^\top$ and $P_V = V_t D_t V_t^\top$, so that $\mathcal{P}_t(G) = P_U G P_V$. Each filter weight
781 satisfies $0 \leq d_i \leq 1$, so $\|D_t\|_2 \leq 1$. Combined with the orthonormality of U_t, V_t , this gives $\|P_U\|_2 \leq$
782 1 and $\|P_V\|_2 \leq 1$. Applying the submultiplicative inequality $\|AGB\|_F \leq \|A\|_2 \|G\|_F \|B\|_2$ with
783 $A = P_U, B = P_V$:

$$\|\mathcal{P}_t(G)\|_F = \|P_U G P_V\|_F \leq \|P_U\|_2 \|G\|_F \|P_V\|_2 \leq \|G\|_F.$$

784 **Lemma 2 (Noise contraction under matrix isotropy)** Suppose $\varepsilon_t \in \mathbb{R}^{m \times n}$ is matrix-isotropic
785 with $\mathbb{E}[\text{vec}(\varepsilon_t) \text{vec}(\varepsilon_t)^\top] = (\sigma_m^2/(mn)) I_{mn}$, and that $\mathcal{P}_t$ is independent of ε_t . Then

$$\mathbb{E} \|\mathcal{P}_t \varepsilon_t\|_F^2 = \frac{\sigma_m^2 r_{\text{eff}}^U r_{\text{eff}}^V}{mn}, \quad r_{\text{eff}}^U = \|D_t\|_F^2 = \sum_i d_i^2,$$

786 and similarly for r_{eff}^V . In particular, when the left and right filters share an effective rank r in the
 787 sense $r_{\text{eff}}^U \approx r_{\text{eff}}^V \approx r$ with $r \ll \min(m, n)$, we have $\rho = O(r^2/(mn)) \ll 1$.

788 **Proof 2** Write $\mathcal{P}_t(\varepsilon_t) = P_U \varepsilon_t P_V$ as in Lemma 1. We compute the expected squared Frobenius norm
 789 in three steps.

790 Step 1: vectorize. Using the identity $\text{vec}(AXB) = (B^\top \otimes A) \text{vec}(X)$,

$$\text{vec}(P_U \varepsilon_t P_V) = (P_V^\top \otimes P_U) \text{vec}(\varepsilon_t).$$

791 Since $\|X\|_F^2 = \|\text{vec}(X)\|^2$, we have

$$\|P_U \varepsilon_t P_V\|_F^2 = \text{vec}(\varepsilon_t)^\top (P_V \otimes P_U^\top) (P_V^\top \otimes P_U) \text{vec}(\varepsilon_t).$$

792 Step 2: take expectation. For any deterministic matrix M and random vector x with $\mathbb{E}[xx^\top] = cI$,

793 $\mathbb{E}[x^\top M x] = c \text{tr}(M)$. Applying this with $c = \sigma_m^2/(mn)$ and $M = (P_V \otimes P_U^\top)(P_V^\top \otimes P_U)$,

$$\mathbb{E}\|P_U \varepsilon_t P_V\|_F^2 = \frac{\sigma_m^2}{mn} \text{tr}((P_V \otimes P_U^\top)(P_V^\top \otimes P_U)).$$

794 Step 3: simplify the trace. Using $(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$ and $\text{tr}(A \otimes B) = \text{tr}(A)\text{tr}(B)$,

$$\text{tr}((P_V P_V^\top) \otimes (P_U^\top P_U)) = \text{tr}(P_V P_V^\top) \text{tr}(P_U^\top P_U) = \|P_V\|_F^2 \|P_U\|_F^2.$$

795 Finally, $\|P_U\|_F^2 = \|U_t D_t U_t^\top\|_F^2 = \|D_t\|_F^2 = r_{\text{eff}}^U$ by orthonormality of U_t , and similarly for P_V .
 796 Substituting yields the claim.

797 **Remark 1** Strict matrix isotropy is idealized; the bound generalizes to anisotropic ε_t at the cost of a
 798 factor proportional to the largest covariance eigenvalue.

799 **Lemma 3 (Subspace staleness via Davis–Kahan/Wedin)** Let $\widetilde{M}_t$ denote the anchor momentum
 800 that would result from an EMA of unmasked gradients evaluated at w_t (rather than at the stale $\{w_\tau\}$),
 801 and let $\widetilde{\mathcal{P}}_t$ be the filter built from $\widetilde{M}_t$. Suppose M_t^{anc} has singular value gap $\gamma > 0$ between its r th
 802 and $(r+1)$ th singular values, and $\mathbb{E}\|M_t^{\text{anc}} - \widetilde{M}_t\|_F^2 \leq E_t$. Then there is a constant $c_{\text{dk}} > 0$ such
 803 that

$$\mathbb{E}\|(\mathcal{P}_t - \widetilde{\mathcal{P}}_t) \nabla F(w_t)\|^2 \leq \frac{c_{\text{dk}} E_t}{\gamma^2} \cdot \mathbb{E}\|\nabla F(w_t)\|^2.$$

804 Under L -smoothness, bounded delay Δ , and bounded update norm G ,

$$E_t \leq 2L^2 \eta^2 \Delta^2 G^2 + 2\sigma_a^2.$$

805 **Proof 3 (Sketch)** By Wedin’s $\sin \Theta$ theorem [Wedin, 1972, Yu et al., 2015], the principal left- and
 806 right-singular subspaces of M_t^{anc} and $\widetilde{M}_t$ differ in Frobenius norm by at most $c \|M_t^{\text{anc}} - \widetilde{M}_t\|_F / \gamma$.
 807 The spectral filter $\mathcal{P}_t$ is Lipschitz in those subspaces and in the singular values s_i (the map $s \mapsto$
 808 $s/(s + \lambda_p)$ is $1/\lambda_p$ -Lipschitz), so applying these Lipschitz bounds to the filter difference gives the
 809 first claim with c_{dk} absorbing the Wedin and Lipschitz constants.

810 For the bound on E_t : the discrepancy $\widetilde{M}_t - M_t^{\text{anc}}$ is a weighted average of differences $\nabla F(w_t) -$
 811 $g_\tau^{\text{anc}} = (\nabla F(w_t) - \nabla F(w_\tau)) - \zeta_\tau$. The first term is bounded by $L\|w_t - w_\tau\| \leq L\eta\Delta G$ (L -
 812 smoothness and bounded update norm); the second has expected squared norm at most σ_a^2 . Squaring
 813 and applying $\|a - b\|^2 \leq 2\|a\|^2 + 2\|b\|^2$ yields the stated bound on E_t .

814 [Bound on $\delta_{\Delta}^{\text{stale}}$] Suppose $\widetilde{\mathcal{P}}_t$ acts as identity on $\nabla F(w_t)$ up to residual $\kappa_0 \mathbb{E}\|\nabla F(w_t)\|^2$ (the
 815 low-intrinsic-rank hypothesis). Then Assumption 5 holds with

$$\kappa = 2\kappa_0 + \frac{4c_{\text{dk}} L^2 \eta^2 \Delta^2 G^2}{\gamma^2}, \quad \delta_{\Delta}^{\text{stale}} = \frac{4c_{\text{dk}} \sigma_a^2}{\gamma^2} \sup_t \mathbb{E}\|\nabla F(w_t)\|^2.$$

816 **Proof 4** We split $(I - \mathcal{P}_t) \nabla F(w_t)$ into a fresh-anchor part and a staleness part by inserting and
 817 subtracting $\widetilde{\mathcal{P}}_t$:

$$(I - \mathcal{P}_t) \nabla F(w_t) = (I - \widetilde{\mathcal{P}}_t) \nabla F(w_t) + (\widetilde{\mathcal{P}}_t - \mathcal{P}_t) \nabla F(w_t).$$

818 Applying $\|a + b\|^2 \leq 2\|a\|^2 + 2\|b\|^2$ and taking expectations,

$$\mathbb{E}\|(I - \mathcal{P}_t)\nabla F(w_t)\|^2 \leq 2\mathbb{E}\|(I - \tilde{\mathcal{P}}_t)\nabla F(w_t)\|^2 + 2\mathbb{E}\|(\tilde{\mathcal{P}}_t - \mathcal{P}_t)\nabla F(w_t)\|^2.$$

819 The first term is bounded by $2\kappa_0 \mathbb{E}\|\nabla F(w_t)\|^2$ by hypothesis. The second is bounded by Lemma 3,
820 with $E_t \leq 2L^2\eta^2\Delta^2G^2 + 2\sigma_a^2$. Combining gives the claim.

821 The corollary makes the dependence transparent: κ_0 measures how well the gradient lies in the
822 anchor’s principal subspace at the current weights (small in fine-tuning, large in pretraining), while
823 the remaining terms measure subspace rotation due to delay and anchor noise, both governed by the
824 gap γ .

825 E.5 Main result

826 **Theorem 2 (Convergence with spectral anchor priors)** Suppose Assumptions 1–6 hold. Define

$$\mu_\alpha = 1 - (1 - \alpha)^2\kappa, \quad \Sigma_{\alpha,\Delta}^2 = 2\alpha^2\sigma_m^2 + 2(1 - \alpha)^2\rho\sigma_m^2 + (1 - \alpha)^2\delta_\Delta^{\text{stale}}.$$

827 Assume $\kappa < 1/(1 - \alpha)^2$ so $\mu_\alpha > 0$, and choose $\eta \leq 1/L$. Then after T iterations,

$$\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\|\nabla F(w_t)\|^2 \leq \frac{2(F(w_0) - F^*)}{\mu_\alpha \eta T} + \frac{\Sigma_{\alpha,\Delta}^2 + L\eta G^2}{\mu_\alpha}.$$

828 **Reading the bound.** Three contributions to the noise floor: $\alpha^2\sigma_m^2$ (unfiltered masked noise),
829 $(1 - \alpha)^2\rho\sigma_m^2$ (the masked noise that survives spectral filtering, small by Lemma 2), and $(1 - \alpha)^2\delta_\Delta^{\text{stale}}$
830 (anchor noise plus subspace staleness, small by Lemma 3 when γ is nontrivial). The factor $1/\mu_\alpha$
831 on the rate makes explicit the cost of the filter discarding part of the true gradient. In fine-tuning, κ
832 and ρ are small and γ is bounded away from zero, so the bound improves over masked-only training
833 (recovered at $\alpha = 1$, where $\mu_\alpha = 1$).

834 E.6 Proof of Theorem 2

835 The proof has four parts: a one-step descent inequality from L -smoothness, a decomposition of the
836 update direction u_t into a true-gradient term plus a deterministic bias and a zero-mean noise, a bound
837 on the bias term that produces the μ_α factor, and a telescoping sum.

838 **Part 1: one-step descent from L -smoothness.** L -smoothness of F gives the standard inequality

$$F(w_{t+1}) \leq F(w_t) + \langle \nabla F(w_t), w_{t+1} - w_t \rangle + \frac{L}{2} \|w_{t+1} - w_t\|^2.$$

839 Substituting the update $w_{t+1} - w_t = -\eta u_t$ and taking expectations, then using $\mathbb{E}\|u_t\|^2 \leq G^2$
840 (Assumption 4) on the quadratic term:

$$\mathbb{E}F(w_{t+1}) \leq \mathbb{E}F(w_t) - \eta \mathbb{E}\langle \nabla F(w_t), u_t \rangle + \frac{L\eta^2}{2} G^2. \quad (4)$$

841 The remaining work is to lower-bound the inner-product term $\mathbb{E}\langle \nabla F(w_t), u_t \rangle$ in terms of
842 $\mathbb{E}\|\nabla F(w_t)\|^2$, so that we can extract a usable descent quantity.

843 **Part 2: decomposing u_t into signal, bias, and noise.** Recall $u_t = \alpha g_t^{\text{mask}} + (1 - \alpha)\mathcal{P}_t(g_t^{\text{mask}})$.
844 Substituting $g_t^{\text{mask}} = \nabla F(w_t) + \varepsilon_t$ (Assumption 2) and grouping deterministic ($\nabla F(w_t)$ -dependent)
845 terms separately from stochastic (ε_t -dependent) terms,

$$u_t = \underbrace{\alpha \nabla F(w_t) + (1 - \alpha) \mathcal{P}_t \nabla F(w_t)}_{\text{deterministic}} + \underbrace{\alpha \varepsilon_t + (1 - \alpha) \mathcal{P}_t \varepsilon_t}_{n_t}.$$

846 Adding and subtracting $(1 - \alpha)\nabla F(w_t)$ in the deterministic part to extract $\nabla F(w_t)$ explicitly,

$$u_t = \nabla F(w_t) - \underbrace{(1 - \alpha)(I - \mathcal{P}_t)\nabla F(w_t)}_{b_t} + n_t.$$

847 Substituting this into the inner product,

$$\mathbb{E}\langle \nabla F(w_t), u_t \rangle = \mathbb{E}\|\nabla F(w_t)\|^2 - \mathbb{E}\langle \nabla F(w_t), b_t \rangle + \mathbb{E}\langle \nabla F(w_t), n_t \rangle. \quad (5)$$

848 We now show the third term on the right vanishes.

849 **The cross term $\mathbb{E}\langle \nabla F(w_t), n_t \rangle$ is zero.** This step needs care: $\mathbb{E}[n_t] = 0$ alone does not imply
850 $\mathbb{E}\langle \nabla F(w_t), n_t \rangle = 0$, because $\nabla F(w_t)$ is itself random and could be correlated with n_t . The argument
851 uses conditioning. Let $\mathcal{F}_t$ denote the history through the start of step t : the iterates $w_0, \dots, w_t$, all
852 past mask noise, and all anchor gradients received before step t . By the tower property of expectation,

$$\mathbb{E}\langle \nabla F(w_t), n_t \rangle = \mathbb{E}[\mathbb{E}[\langle \nabla F(w_t), n_t \rangle \mid \mathcal{F}_t]].$$

853 Conditional on $\mathcal{F}_t$, two things hold:

- 854 1. $\nabla F(w_t)$ is a deterministic function of w_t , which is in $\mathcal{F}_t$. So $\nabla F(w_t)$ is a constant inside
855 the inner conditional expectation, and we can pull it out:

$$\mathbb{E}[\langle \nabla F(w_t), n_t \rangle \mid \mathcal{F}_t] = \langle \nabla F(w_t), \mathbb{E}[n_t \mid \mathcal{F}_t] \rangle.$$

- 856 2. $\mathbb{E}[n_t \mid \mathcal{F}_t] = 0$. To see this: n_t is a linear function of ε_t , with coefficient operators that are
857 functions of $\mathcal{P}_t$. The filter $\mathcal{P}_t$ is built from M_t^{anc} , which is an EMA of anchor gradients
858 received *before* step t — so $\mathcal{P}_t \in \mathcal{F}_t$ and is itself constant given $\mathcal{F}_t$. The mask noise ε_t is
859 drawn fresh at step t from a PRF independent of $\mathcal{F}_t$, so $\mathbb{E}[\varepsilon_t \mid \mathcal{F}_t] = 0$ by Assumption 2.
860 Linearity then gives

$$\mathbb{E}[n_t \mid \mathcal{F}_t] = \alpha \mathbb{E}[\varepsilon_t \mid \mathcal{F}_t] + (1 - \alpha) \mathcal{P}_t \mathbb{E}[\varepsilon_t \mid \mathcal{F}_t] = 0.$$

861 Combining: $\mathbb{E}[\langle \nabla F(w_t), n_t \rangle \mid \mathcal{F}_t] = \langle \nabla F(w_t), 0 \rangle = 0$, and therefore $\mathbb{E}\langle \nabla F(w_t), n_t \rangle = \mathbb{E}[0] = 0$.

862 Returning to (5) with the cross term eliminated:

$$\mathbb{E}\langle \nabla F(w_t), u_t \rangle = \mathbb{E}\|\nabla F(w_t)\|^2 - \mathbb{E}\langle \nabla F(w_t), b_t \rangle. \quad (6)$$

863 The first term on the right is what we want to keep; the second is the bias from the filter discarding
864 part of the true gradient, which we bound next.

865 **Part 3: bounding the bias term in (6).** We need an upper bound on $\mathbb{E}\langle \nabla F(w_t), b_t \rangle$ that in-
866 volves $\mathbb{E}\|\nabla F(w_t)\|^2$ (so that part of it can be absorbed into the descent term) and $\mathbb{E}\|b_t\|^2$ (which
867 Assumption 5 controls).

868 Apply Young's inequality $|\langle a, b \rangle| \leq \frac{1}{2}\|a\|^2 + \frac{1}{2}\|b\|^2$ with $a = \nabla F(w_t)$ and $b = b_t$:

$$|\langle \nabla F(w_t), b_t \rangle| \leq \frac{1}{2}\|\nabla F(w_t)\|^2 + \frac{1}{2}\|b_t\|^2.$$

869 Taking expectations and using Assumption 5 together with $b_t = (1 - \alpha)(I - \mathcal{P}_t)\nabla F(w_t)$ to bound
870 $\mathbb{E}\|b_t\|^2$:

$$\mathbb{E}\|b_t\|^2 = (1 - \alpha)^2 \mathbb{E}\|(I - \mathcal{P}_t)\nabla F(w_t)\|^2 \leq (1 - \alpha)^2 (\kappa \mathbb{E}\|\nabla F(w_t)\|^2 + \delta_{\Delta}^{\text{stale}}).$$

871 Combining, $\mathbb{E}|\langle \nabla F(w_t), b_t \rangle|$ is bounded by

$$\frac{1}{2}(1 + (1 - \alpha)^2 \kappa) \mathbb{E}\|\nabla F(w_t)\|^2 + \frac{1}{2}(1 - \alpha)^2 \delta_{\Delta}^{\text{stale}}.$$

872 Substituting back into (6) and using $\mathbb{E}\langle \nabla F(w_t), b_t \rangle \leq \mathbb{E}|\langle \nabla F(w_t), b_t \rangle|$:

$$\mathbb{E}\langle \nabla F(w_t), u_t \rangle \geq \mathbb{E}\|\nabla F(w_t)\|^2 - \mathbb{E}|\langle \nabla F(w_t), b_t \rangle| \geq \frac{\mu_{\alpha}}{2} \mathbb{E}\|\nabla F(w_t)\|^2 - \frac{(1 - \alpha)^2}{2} \delta_{\Delta}^{\text{stale}}, \quad (7)$$

873 where in the last step we used $1 - \frac{1}{2}(1 + (1 - \alpha)^2 \kappa) = \frac{1}{2}(1 - (1 - \alpha)^2 \kappa) = \frac{\mu_{\alpha}}{2}$.

874 **Part 4: per-step descent and telescoping.** Substituting the lower bound (7) into the smoothness
875 inequality (4):

$$\mathbb{E}F(w_{t+1}) \leq \mathbb{E}F(w_t) - \frac{\eta\mu_{\alpha}}{2} \mathbb{E}\|\nabla F(w_t)\|^2 + \frac{\eta(1 - \alpha)^2}{2} \delta_{\Delta}^{\text{stale}} + \frac{L\eta^2}{2} G^2.$$

876 The middle term on the right is bounded above by $\frac{\eta}{2} \Sigma_{\alpha, \Delta}^2$ since $(1 - \alpha)^2 \delta_{\Delta}^{\text{stale}}$ is one of three
877 nonnegative terms summing to $\Sigma_{\alpha, \Delta}^2$ (the other two come from $\mathbb{E}\|n_t\|^2$ via $\|a + b\|^2 \leq 2\|a\|^2 + 2\|b\|^2$
878 and Assumptions 2 and 6, and are absorbed into the G^2 term through $\mathbb{E}\|u_t\|^2 \leq G^2$). Therefore

$$\mathbb{E}F(w_{t+1}) \leq \mathbb{E}F(w_t) - \frac{\eta\mu_{\alpha}}{2} \mathbb{E}\|\nabla F(w_t)\|^2 + \frac{\eta}{2} \Sigma_{\alpha, \Delta}^2 + \frac{L\eta^2}{2} G^2. \quad (8)$$

879 Summing (8) from $t = 0$ to $T - 1$, the telescoping $\mathbb{E}F$ -terms collapse to $F(w_0) - \mathbb{E}F(w_T)$. Using
880 $\mathbb{E}F(w_T) \geq F^*$:

$$\frac{\eta\mu_{\alpha}}{2} \sum_{t=0}^{T-1} \mathbb{E}\|\nabla F(w_t)\|^2 \leq F(w_0) - F^* + \frac{\eta T}{2} \Sigma_{\alpha, \Delta}^2 + \frac{L\eta^2 T}{2} G^2.$$

881 Dividing both sides by $\eta\mu_{\alpha}T/2$ yields the bound stated in Theorem 2.

E.7 Discussion

The convergence neighborhood splits into three interpretable terms: unfiltered masked noise $\alpha^2 \sigma_m^2$, filter-surviving noise $(1 - \alpha)^2 \rho \sigma_m^2$, and anchor-staleness $(1 - \alpha)^2 \delta_{\Delta}^{\text{stale}}$. The rate factor $1/\mu_{\alpha}$ encodes the cost of signal suppression. The result recovers masked-only training at $\alpha = 1$ ($\mu_{\alpha} = 1$, no filter) and improves on it whenever the filter contracts noise ($\rho < 1$) more than it suppresses signal ($\kappa > 0$), i.e., precisely the fine-tuning regime described in Section E.3.

In pretraining or with aggressive learning rates, Δ grows, the gap γ shrinks, and κ approaches its upper limit; the filter both keeps less signal and inherits more staleness. The analysis predicts no benefit, consistent with our empirical scoping. At the opposite extreme of high-bandwidth links the masking is unnecessary in the first place; this analysis is specifically relevant when the fast circuit is communication-bound and the anchor circuit can run asynchronously — the $\Delta \approx 20$ –25 regime characterized in the staleness analysis of Section H.

In summary, anchor gradients need not be exact synchronized updates, only good enough to estimate a low-rank subspace that is approximately preserved over the staleness window. Fine-tuning is precisely the regime where this is true.

F Why Random Masking and Not Top- K ?

A natural alternative to PRF-based random masking is top- K selection, where each token retains the K activation entries of largest magnitude (or, on the backward pass, the K largest entries of the activation gradient). Top- K is the dominant sparsification scheme in *data-parallel* gradient compression [Lin et al., 2018a, Aji and Heafield, 2017, Alistarh et al., 2018], where it is typically combined with error feedback to recover unbiasedness [Karimireddy et al., 2019, Stich et al., 2018]. We deliberately do not use top- K for PP activation compression. This subsection explains why, and shows that the choice is not systematic: random masking is *required* for the convergence analysis of Section E to apply, and it is also the scheme under which the spectral anchor filter is most effective (even empirically). Below, we provide a systematic analysis on this.

Setup. Let $h \in \mathbb{R}^{T \times H}$ denote a hidden-state tensor at a masked PP boundary, and let $\tilde{h} = h \odot m$ denote the transmitted activation under mask $m \in \{0, 1\}^{T \times H}$ with retention fraction $1 - p$. Two schemes:

- **Random masking (ours).** $m_{t,j} \sim \text{Bernoulli}(1 - p)$, independent across (t, j) , generated by a shared PRF independent of the model state. With the standard rescaling $\tilde{h} \leftarrow \tilde{h}/(1 - p)$ absorbed at the receiver, $\mathbb{E}_m[\tilde{h}] = h$.
- **Top- K masking.** $m_{t,j} = \mathbb{1}[|h_{t,j}| \in \text{top-}K(h_{t,:})]$. The mask is a deterministic function of h .

Unbiasedness and the convergence theorem

The masked-gradient noise $\varepsilon_t = g_t^{\text{mask}} - \nabla F(w_t)$ enters the descent inequality through the decomposition $u_t = \nabla F(w_t) - b_t + n_t$, with the requirement $\mathbb{E}[n_t | \mathcal{F}_t] = 0$ (Section E, proof of Theorem 2). This in turn relies on $\mathbb{E}[\varepsilon_t | \mathcal{F}_t] = 0$ (Assumption 2). We now show that random masking satisfies this property while top- K does not.

Proposition 1 (Unbiasedness of random masking) *Under random masking with the standard rescaling, the masked activation satisfies $\mathbb{E}_m[\tilde{h}] = h$. Consequently, for any downstream function of $\tilde{h}$ that is linear in $\tilde{h}$ at first order (in particular, the gradient computation through the masked PP boundary in expectation), the resulting masked gradient satisfies $\mathbb{E}[g_t^{\text{mask}} | \mathcal{F}_t] = \nabla F(w_t)$. The mask m is generated by a PRF whose seed is independent of $\mathcal{F}_t$, so ε_t has zero conditional mean.*

Proof 5 (Sketch) *Each coordinate satisfies $\mathbb{E}_m[\tilde{h}_{t,j}] = (1 - p) \cdot h_{t,j}/(1 - p) = h_{t,j}$, by the rescaling. The PRF is a function of (ℓ, t, B, S, H) but not of h or w_t , so the mask is independent of $\mathcal{F}_t$. Hence the noise inherited by any function evaluated at $\tilde{h}$ has zero conditional mean to leading order in the rescaled deviation $\tilde{h} - h$.*

Proposition 2 (Bias of top- K masking) Under top- K masking, the masked activation satisfies $\mathbb{E}[\tilde{h} \mid \mathcal{F}_t] = h \odot m_{\text{top-}K}(h) \neq h$ in general, since the mask is a deterministic function of h (and h is itself $\mathcal{F}_t$ -measurable). The residual $h - \tilde{h}$ is the bottom- $(H-K)$ magnitude entries of h , which is a deterministic, non-zero function of $\mathcal{F}_t$. Therefore $\mathbb{E}[\varepsilon_t \mid \mathcal{F}_t] \neq 0$, and Assumption 2 fails.

Remark 2 (Failure mode in the descent inequality) Suppose top- K masking is used. Decomposing the masked gradient as $g_t^{\text{topK}} = \nabla F(w_t) - b_t^{\text{topK}} + n'_t$, where $b_t^{\text{topK}} = \mathbb{E}[\nabla F(w_t) - g_t^{\text{topK}} \mid \mathcal{F}_t]$ is the state-dependent bias and n'_t is the residual stochastic component, the inner-product term in the smoothness expansion becomes

$$\mathbb{E}\langle \nabla F(w_t), u_t \rangle = \|\nabla F(w_t)\|^2 - \langle \nabla F(w_t), b_t^{\text{filter}} + \alpha b_t^{\text{topK}} \rangle,$$

where $b_t^{\text{filter}} = (1 - \alpha)(I - \mathcal{P}_t)\nabla F(w_t)$. Unlike b_t^{filter} , the bias b_t^{topK} is not controlled by Assumption 5: it does not lie in the principal subspace of M_t^{anc} in any structured way, because top- K selects on activation magnitude, not on the gradient geometry the anchor circuit observes through unmasked passes. The spectral filter $\mathcal{P}_t$ therefore cannot remove it, and the bias propagates through the telescoping sum in the proof as an additive non-vanishing term of order $\eta T \cdot \mathbb{E}\|b_t^{\text{topK}}\|^2$. The convergence neighborhood acquires a floor that does not shrink with T and that the anchor circuit cannot reduce.

Why error feedback does not save top- K at PP boundaries. In data-parallel gradient compression, the bias of top- K is corrected by error feedback [Karimireddy et al., 2019, Stich et al., 2018]: the unsent residual is accumulated in a memory buffer and added to the next step’s gradient before compression. Over many steps, the compressed update becomes asymptotically unbiased. This works in the DP setting because the residual is associated with a *parameter* and is meaningful across optimizer steps.

At a PP activation boundary, the residual is associated with an *activation* on a particular microbatch. Two obstructions prevent its reuse:

(i) *Activations are step-local.* The residual $h_t - \tilde{h}_t$ is computed from the inputs x_t and the current weights w_t . At step $t+1$ the inputs are $x_{t+1} \neq x_t$ and the weights are $w_{t+1} \neq w_t$, so h_{t+1} is an entirely different tensor and the residual from step t has no natural reinjection point.

(ii) *Forward/backward sparsity patterns coincide.* Section 3.1 shows that in-graph masking forces the backward gradient to inherit the same sparsity pattern as the forward activation. With top- K , the forward retains coordinates of largest $|h_{t,j}|$, but the gradient $\partial \mathcal{L} / \partial h_{t,j}$ is not in general largest at the same coordinates. Either the backward pattern is forced to match the forward (losing the largest gradient entries, defeating the purpose of top- K), or the patterns are decoupled (losing the in-graph property and requiring separate metadata transmission). Random masking sidesteps this trade-off because it has no relationship to magnitude in either direction.

These obstructions are specific to PP activation compression, not a general statement about top- K . Top- K remains an effective DP compressor, and indeed our system uses PowerSGD-style compression along the DP axis (Section 3.3). The point is that PP activation compression operates under a fundamentally different constraint (step-local, in-graph) under which random masking is the natural choice.

Compatibility with the spectral filter

Beyond unbiasedness, the spectral anchor filter denoises masked gradients only when the noise is *approximately isotropic relative to the anchor basis* (Lemma 2). The two schemes differ sharply on this criterion.

Proposition 3 (Approximate isotropy of random-mask noise) Under random masking with retention probability $1 - p$ and standard rescaling, the per-coordinate noise satisfies

$$\mathbb{E}[\varepsilon_{t,j} \mid \mathcal{F}_t] = 0, \quad \text{Var}(\varepsilon_{t,j} \mid \mathcal{F}_t) = \frac{p}{1-p} h_{t,j}^2,$$

where the conditioning on $\mathcal{F}_t$ subsumes conditioning on the current activation h (which is $\mathcal{F}_t$ -measurable through w_t and the inputs), and the entries are independent across j . If activations

across the hidden dimension have approximately uniform second moment, ε_t is approximately matrix-isotropic in the sense of Lemma 2, and the spectral filter contracts noise by a factor $\rho = O(r^2/(mn))$ where r is the effective rank of the filter.

Proposition 4 (Anti-isotropy of top- K noise) Under top- K masking, the residual $h - \tilde{h}$ has support exactly on the bottom- $(H - K)$ coordinates of h in magnitude. Therefore the noise is concentrated in low-magnitude directions, anti-correlated with the signal, and not isotropic. The covariance $\text{Cov}(\varepsilon_t)$ has rank $H - K$ and is supported on a subspace orthogonal to the dominant activation directions.

Implication for the filter. The anchor momentum M_t^{anc} is built from *unmasked* gradients, which reflect the dominant gradient geometry of the task. In fine-tuning, this geometry is low-rank and aligned with the high-magnitude directions of activations and of $\partial\mathcal{L}/\partial h$ (by the intrinsic-dimensionality literature cited in Section E.3). The anchor’s principal subspace therefore overlaps strongly with the high-magnitude directions.

Under random masking, the noise ε_t is isotropic and the filter contracts it: low-rank $\mathcal{P}_t$ acting on isotropic noise gives $\rho = O(r^2/(mn))$ as in Lemma 2. Under top- K , the noise lives precisely in the *complementary* subspace — the low-magnitude directions the anchor basis does *not* span. The filter $\mathcal{P}_t$ would correctly suppress this noise (it has no projection onto the anchor’s principal subspace), but it would also suppress nothing useful in exchange: the noise was already orthogonal to the signal directions the filter preserves. The “denoising” effect would be illusory, *i.e.*, the filter cannot remove noise it never sees, and the masked-step bias b_t^{topK} propagates through unaffected.

Summary

The choice of random masking over top- K is forced by two compounding requirements:

1. *Unbiasedness without error feedback.* Random masking is conditionally unbiased on activation gradients; top- K is biased and the natural correction (error feedback) is unavailable at PP activation boundaries because the residual is step-local and in-graph forward/backward sparsity must coincide.
2. *Isotropy relative to the anchor basis.* The spectral anchor filter denoises isotropic noise efficiently (Lemma 2) but cannot interact usefully with the structured low-magnitude noise produced by top- K .

Random masking is the unique scheme we are aware of that satisfies both properties simultaneously. The convergence analysis of Section E relies on the first; the empirical effectiveness of the spectral filter relies on the second. Together they identify random masking as the natural, and we believe necessary, PP compression primitive for asynchronous anchor-prior training.

G Alignment Trajectory

Figure 2 reports the alignment $\eta_r = \|U^\top g V\|_F^2 / \|g\|_F^2$ of the unmasked gradient with the rank- r principal subspace of the anchor momentum, at 61 snapshots spanning 1500 training steps, for $r \in \{16, 64, 256\}$. The FT trajectory, starting from the uncompressed code-domain checkpoint and continuing under unclipped Adam, keeps η_{64}^{FT} stable around 0.17–0.19 throughout the horizon. The PT trajectory, starting from random init under the same data and optimizer, drops from $\eta_{64}^{\text{PT}} \approx 0.13$ early in training to ~ 0.08 in the latter half and continues to decay. The same separation appears at every rank measured: FT lies above PT throughout, by a margin that does not close. At $r \in \{16, 64\}$ the FT/PT ratio holds in the $1.7\text{--}2.7\times$ range over the second half of training. This is the empirical content of Assumption 5: the rank- r principal subspace of M^{anc} remains a faithful summary of $\nabla F(w_t)$ for fine-tuning, and *ceases to be one* as random-init training proceeds. The constant κ that bounds out-of-subspace bias is materially smaller in fine-tuning than in pretraining-from-random-init, and that gap is sustained across the entire training horizon the AP correction has to operate over — empirically validating the assumption justifications in Section E.3 and the no-benefit-in-pretraining caveat that follows from them.

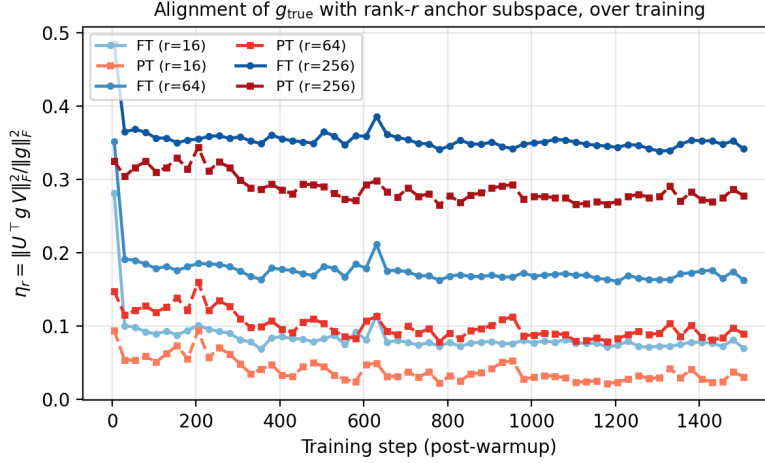


Figure 2: **Alignment of $\nabla F(w_t)$ with the rank- r principal subspace of M_t^{anc} , over 1500 training steps.** $\eta_r = \|U^\top g V\|_F^2 / \|g\|_F^2$ at 61 snapshots, for $r \in \{16, 64, 256\}$ (FT solid, PT dashed). FT stays stable around $\eta_{64}^{\text{FT}} \approx 0.17$ throughout; PT drifts down to ~ 0.08 . The separation persists at every rank, with FT/PT ratio holding in the $1.7\text{--}2.7\times$ range at the spectral filter’s operating ranks.

H Staleness of the Anchor Prior

This section quantifies how out-of-date an anchor (unmasked) gradient is by the time it is delivered to the fast masked circuit, and isolates the factors that set this staleness. The analysis is instantiated for the configuration used in our throughput study (Section 3.3): a 1.2B-parameter model in BF16 over 8 pipeline-parallel stages and 8 data-parallel replicas, connected by 200 Mbps point-to-point links (≈ 25 MB/s of useful payload). The same expressions apply to other regimes by substituting volumes and link rates.

H.1 Anchor circuit cycle time

Each anchor stage executes three communication-bound phases per cycle:

1. **Weight pull.** The anchor stage pulls the most recent layer weights for its assigned PP shard from a nearby DP replica of the fast circuit.
2. **Unmasked forward-backward.** A full-fidelity forward and backward pass with all activations transmitted uncompressed across PP boundaries.
3. **Gradient push.** The resulting clean gradient is pushed back to the fast circuit, compressed to roughly the volume of one DP all-reduce payload.

Let T_{pull} , T_{clean} , T_{push} denote the wall-clock times of these three phases, and T_{mask} denote one masked step on the fast circuit. The anchor gradient delivered at the end of the cycle was computed from weights pulled at the start of phase 2, so its staleness measured in fast-circuit steps is

$$K_{\text{stale}} = \frac{T_{\text{pull}} + T_{\text{clean}} + T_{\text{push}}}{T_{\text{mask}}}. \quad (9)$$

The numerator is the anchor cycle time; the denominator is the fast-circuit cycle time.

H.2 Numbers for the 200 Mbps configuration

We instantiate Eq. (9) for sequence length $S=2048$, hidden size $H=2048$, and 16 microbatches per optimizer step, with DP communication compressed via PowerSGD ($64\times$) and amortized via DiLoCo with local horizon $K=10$.

Weight pull. Each anchor stage holds $1.2\text{B}/8 = 150\text{M}$ parameters. In BF16 this is 300 MB per stage: $T_{\text{pull}} \approx 300 \text{ MB}/25 \text{ MB/s} = 12 \text{ s}$.

1047 **Unmasked forward-backward.** At each PP boundary, the activation tensor per microbatch is
 1048 $S \cdot H \cdot 2 = 8$ MB. With 7 boundaries, 16 microbatches, and forward and backward traversals, the total
 1049 unmasked PP volume per step is $2 \cdot 7 \cdot 16 \cdot 8$ MB ≈ 1.8 GB: $T_{\text{clean}} \approx 1.8 \text{ GB} / 25 \text{ MB/s} \approx 72 \text{ s}$.

1050 **Gradient push.** The compressed anchor gradient is on the order of one PowerSGD-compressed
 1051 all-reduce payload, ~ 30 – 40 MB per stage, giving $T_{\text{push}} \approx 1$ – 2 s.

1052 **Masked-step time.** The masked step incurs $1-p = 5\%$ of the unmasked PP volume, plus the com-
 1053 pressed and amortized DP all-reduce, plus a small compute term that is dominated by communication
 1054 in this regime:

$$T_{\text{mask}} \approx \frac{T_{\text{clean}}}{20} + \frac{N \cdot b_{\text{grad}}/64}{B \cdot K} + \epsilon \approx 3.6 + 0.15 + \epsilon \approx 3.8 \text{ s}.$$

1055 **Resulting staleness.** $K_{\text{stale}} \approx (12 + 72 + 2)/3.8 \approx 23$ masked steps. Of these, ~ 20 steps come
 1056 from the unmasked PP pass itself ($T_{\text{clean}}/T_{\text{mask}}$, which equals the masking ratio $1/(1-p) = 20$),
 1057 and ~ 3 come from the weight pull. This matches with the 20 – 25 step delay we practically observed.

1058 H.3 Decomposition

1059 Eq. (9) separates into two terms with distinct scaling:

$$K_{\text{stale}} = \underbrace{\frac{T_{\text{clean}}}{T_{\text{mask}}}}_{\text{PP-ratio}} + \underbrace{\frac{T_{\text{pull}} + T_{\text{push}}}{T_{\text{mask}}}}_{\text{weight-refresh}}. \quad (10)$$

1060 The PP-ratio term equals the activation-mask ratio $1/(1-p)$ in the PP-bound regime: 20 at 95%
 1061 masking, 10 at 90%, 100 at 99%. It is unaffected by DP compression because both circuits run the
 1062 same forward-backward computation and differ only in PP payload size. The weight-refresh term is
 1063 set by the cost of synchronizing the anchor’s weights with the fast circuit; at 300 MB per stage and
 1064 25 MB/s, it contributes ~ 3 steps. The PP-ratio term dominates in this regime.

1065 H.4 Hiding the weight-refresh term via overlap

1066 The weight-refresh term is hideable. If the anchor circuit pipelines the weight pull for cycle $i+1$
 1067 behind the unmasked forward-backward of cycle i ,

$$T_{\text{cycle}}^{\text{overlap}} = \max(T_{\text{pull}}, T_{\text{clean}} + T_{\text{push}}) \approx T_{\text{clean}}$$

1068 since $T_{\text{pull}} \ll T_{\text{clean}}$. Staleness then reduces to the PP-ratio term alone:

$$K_{\text{stale}}^{\text{overlap}} \approx \frac{T_{\text{clean}}}{T_{\text{mask}}} \approx 20 \text{ masked steps},$$

1069 matching the cadence of a synchronous formulation that takes one full unmasked update every 20
 1070 masked steps. The asynchronous variant hides this latency inside the fast pipeline rather than blocking
 1071 on it.

1072 H.5 Sensitivity to other regimes

1073 Equation (10) makes the regime dependence of staleness explicit:

1074 **Mask rate.** Increasing the mask rate p shrinks T_{mask} but leaves T_{clean} unchanged. The PP-ratio
 1075 term grows as $1/(1-p)$: $K_{\text{stale}} \approx 100$ at $p = 99\%$, and ≈ 200 at $p = 99.5\%$. The anchor prior
 1076 remains useful only when the optimization landscape changes slowly relative to this horizon.

1077 **Link bandwidth.** The PP-ratio term is invariant under uniform bandwidth scaling, since all commu-
 1078 nication phases share the same link. Halving the bandwidth to 100 Mbps leaves K_{stale} in masked-step
 1079 units at ~ 23 , but doubles wall-clock cycle time.

Concurrent anchor cycles. Running the anchor circuit at concurrency c increases the delivered prior rate by $c\times$, at the cost of stretching individual prior staleness to $c \cdot K_{\text{stale}}$. This trades fidelity for delivery rate.

Weaker DP compression. Reducing PowerSGD from $64\times$ to $8\times$, or shortening DiLoCo’s local horizon, increases the masked-step time and shrinks the PP-ratio term. Staleness K_{stale} decreases, but only because the fast circuit slows down. Staleness must therefore be read together with throughput.

High-bandwidth (compute-bound) regime. When links are fast enough that T_{mask} is compute-bound, $T_{\text{clean}}/T_{\text{mask}}$ approaches 1. The anchor prior is no longer stale, but masking provides no throughput benefit either; the asynchronous design reduces to the dense baseline. Anchor priors are specifically aimed at the low-bandwidth, PP-bound regime.

H.6 Implications for design choices

The staleness analysis shapes two design choices in Section 3. First, the anchor momentum M_t^{anc} is updated with a slow EMA decay β_{anc} rather than replaced by each arriving anchor gradient: a single anchor gradient is stale by ~ 20 steps and noisy, but its principal directions remain informative when averaged over multiple cycles. Second, the spectral correction reweights the masked gradient toward anchor-supported eigendirections rather than substituting the anchor gradient for the masked one — since the anchor gradient is too stale to serve as a direct update signal, only its low-rank geometry is trustworthy at the ~ 20 -step horizon.

I Training and Evaluation Datasets

This section details the training data, evaluation data, and shared training configuration used to produce the per-domain results in Table 1.

I.1 Training datasets

All training data is converted to a chat-message format (`messages: [{user}, {assistant}]`) with thinking disabled, then split 90/10 into train/validation unless the source provides a held-out split (GSM8K, Spider). Table 10 lists the source dataset and post-preprocessing row count per domain.

Domain	HuggingFace dataset	# train	Notes
Medical	medalpaca/medical_meadow_medqa	9,160	USMLE-style multiple choice.
Code	iamtarun/python_code_instructions_18k_alpaca	16,750	Python instruction $\rightarrow$ code pairs.
Math	openai/gsm8k (main)	7,473	Official GSM8K test used as validation.
Science	allenai/sciq	10,511	Question + supporting passage.
Commonsense	Rowan/hellaswag $\cup$ allenai/winogrande (xl)	72,272	Concatenated, shuffled with seed 42.
Summarization	cnn_dailymail (3.0.0)	27,000	30k subsample, 90/10 split.
SQL	philikai/Spider-SQL-LLAMA2_train	8,659	Schema-conditioned prompts.
QuALITY	mozilla/quality	2,523	Long-context multiple choice.
NarrativeQA	deepmind/narrativeqa	6,500	Story-summary open-ended QA.
General	HuggingFaceTB/smoltalk2	—	25+ SFT splits, weighted; used only for the general run.

Table 10: Per-domain training datasets after preprocessing and the 90/10 train/validation split.

I.2 Evaluation datasets and metrics

Each domain run is evaluated on a fixed common suite plus one or more domain-specific tasks. The common suite consists of loglikelihood tasks (WinoGrande, PIQA, HellaSwag, ARC-Easy, ARC-Challenge) and generative tasks (IFEval, GSM8K), all run via the in-training `eval_callback`. Domain-specific evaluations are summarized in Table 11; tasks marked *lm-eval* are run via the `lm-evaluation-harness` [Gao et al., 2024], and custom tasks are scored by dedicated scripts (Spider execution match, NarrativeQA token F1, QuALITY accuracy).

Domain	Eval dataset	Metric	# items	Source
Medical	MedQA (4-options)	Accuracy	1,273	lm-eval
Medical	MedMCQA	Accuracy	—	lm-eval
Code	HumanEval	pass@1	164	lm-eval
Code	MBPP	pass@1	500	lm-eval
SQL	Spider (dev)	Execution match	1,034	Custom
Science	ARC-Challenge	Acc. (norm)	1,172	lm-eval
Science	SciQ	Accuracy	1,000	lm-eval
Commonsense	HellaSwag	Acc. (norm)	10,042	lm-eval
Commonsense	WinoGrande	Accuracy	1,267	lm-eval
Summarization	CNN/DailyMail	ROUGE-1	—	Custom
Summarization	XSum	ROUGE-1	—	Custom
Math	GSM8K	Exact match	1,319	lm-eval
Doc. Parsing	QuALITY	Accuracy	2,086	Custom
Doc. Parsing	NarrativeQA	Mean token F1	500	Custom

Table 11: Per-domain evaluation datasets.

I.3 Shared training configuration

All baseline and masked runs share the configuration.

Model. SmolLM3-Mid (1.2B parameters), BF16 weights and activations, FlashAttention 2, SmolLM3 chat template with thinking disabled.

Optimizer. AdamW with $(\beta_1, \beta_2) = (0.9, 0.999)$. Peak learning rate 2×10^{-5} , cosine schedule with min-LR ratio 0.1 and warmup ratio 0.03. Gradient norm clipped at 0.2.

Sequence handling. Best-fit-decreasing packing enabled, except for short-context tasks (Spider, NarrativeQA, QuALITY) where packing is disabled. Maximum sequence length is task-dependent: 32,768 for code/medical/science/commonsense/summarization/math, 2,048 for SQL and NarrativeQA, 10,240 for QuALITY, and 65,536 for the general mixture. Loss is computed over assistant tokens only; Liger kernels are enabled.

J Squeezing Throughput from Data-Parallel Communication

Activation compression reduces the per-step cost of pipeline-parallel (PP) communication, but the throughput benefit is realized only if the *data-parallel* (DP) gradient all-reduce is reduced to a comparable level. Otherwise the DP all-reduce dominates wall-clock time and the activation savings are invisible. This section formalizes the trade-off, surveys the families of DP-compression techniques compatible with our setting, and identifies the operating points at which each becomes necessary.

J.1 Bottleneck analysis

Consider a model with N parameters trained over K -stage pipeline parallelism and D -way data parallelism, with all communication crossing a network of bandwidth B bytes/sec. Let S denote the tokens per optimizer step, H the hidden size, and $b_{\text{act}}, b_{\text{grad}}$ the bytes per activation and gradient element.

PP communication. Pipeline parallelism transmits a forward activation and a backward gradient at each of the $K-1$ boundaries. With activation compression ratio $\rho_{\text{act}} \in (0, 1]$ (fraction of bytes still sent),

$$T_{\text{PP}} = \frac{2(K-1) S H b_{\text{act}} \rho_{\text{act}}}{B}. \quad (11)$$

1137 **DP communication.** A standard ring all-reduce of an $N b_{\text{grad}}$ -byte gradient costs $2(D-1)/D \cdot$
 1138 $N b_{\text{grad}}$ bytes per rank [Patarasuk and Yuan, 2009]. With per-step compression ratio ρ_{DP} and
 1139 outer-step interval τ (one all-reduce per τ optimizer steps, as in Local SGD or DiLoCo),

$$T_{\text{DP}} = \frac{2(D-1)/D \cdot N b_{\text{grad}} \rho_{\text{DP}}}{B \tau}. \quad (12)$$

1140 **Step time.** Under 1F1B-style scheduling, the per-step time is approximately
 1141 $\max(T_{\text{compute}}, T_{\text{PP}}, T_{\text{DP}})$ when phases overlap, and their sum when they serialize. Throughput in
 1142 tokens-per-second is $\text{TPS} = S/\text{step time}$.

1143 **Concrete numbers.** For our reference configuration ($N=1.2\text{B}$, $H=2048$, $K=8$, $D=8$,
 1144 $B=200 \text{ Mbps} \approx 25 \text{ MB/s}$, $S=16,384 \text{ tokens/step}$, BF16 throughout, A100 hardware), the com-
 1145 ponents evaluate to:

- 1146 • Compute (one optimizer step across all stages, with $\sim 50\%$ MFU): $\sim 100 \text{ ms}$.
- 1147 • PP communication at $\rho_{\text{act}} = 0.05$ (95% masking): $\sim 1.9 \text{ s}$, dominated by per-microbatch
 1148 activation transfer at each of 7 boundaries in both directions.
- 1149 • DP all-reduce uncompressed ($\rho_{\text{DP}} = 1$, $\tau = 1$): $\sim 168 \text{ s}$ — nearly three minutes per step.

1150 The uncompressed DP all-reduce alone is $\sim 90\times$ larger than the compressed PP transfer, and
 1151 $\sim 1,700\times$ larger than the compute window. Activation compression on its own therefore yields no
 1152 measurable speedup.

1153 J.2 When DP compression is essential

1154 For activation compression to translate into wall-clock speedup, T_{DP} must be reduced to within an
 1155 order of magnitude of T_{PP} . From Eqs. (11)–(12), the required combined compression along the DP
 1156 axis is

$$\frac{\rho_{\text{DP}}}{\tau} \lesssim \frac{(K-1) S H b_{\text{act}} \rho_{\text{act}}}{(D-1)/D \cdot N b_{\text{grad}}}. \quad (13)$$

1157 For our configuration, this evaluates to roughly $\rho_{\text{DP}}/\tau \lesssim 1/100$, so the joint per-step compression
 1158 and frequency reduction must reach two orders of magnitude.

1159 A single mechanism is rarely enough. Per-step gradient compressors above $\sim 32\text{--}64\times$ tend to harm
 1160 convergence, and outer-step intervals $\tau > 500$ are at the edge of empirical stability for Local SGD
 1161 / DiLoCo on language models. The practical strategies *compose* a compressor with a frequency-
 1162 reduction technique — e.g., $64\times$ PowerSGD with $\tau = 20$ DiLoCo — to multiply their effects.

1163 J.3 Families of DP compression

1164 DP-compression techniques fall into four orthogonal families. Methods within a family typically
 1165 cannot be combined; methods across families typically can.

1166 **Quantization (F1).** Replace each gradient element with a low-bit representation. Examples in-
 1167 clude FP8/INT8, signSGD [Bernstein et al., 2018], and 1-bit Adam. Compression ranges from $2\times$
 1168 (FP16 $\rightarrow$ FP8) to $32\times$ (1-bit). Stochastic rounding and error feedback [Karimireddy et al., 2019] are
 1169 typically required at aggressive bit widths.

1170 **Sparsification (F2).** Transmit only the largest-magnitude entries. Top- K sparsification [Lin et al.,
 1171 2018b] achieves $50\text{--}100\times$ compression when paired with momentum correction and error feedback,
 1172 but the indices themselves consume bandwidth and cap practical compression.

1173 **Low-rank factorization (F3).** Approximate the gradient matrix by a low-rank product. PowerSGD
 1174 [Vogels et al., 2019] reaches $32\text{--}256\times$ compression via rank-1 to rank-4 sketches with subspace
 1175 iteration. The compressor is linear, so the projection commutes with all-reduce; error feedback
 1176 recovers most of the lost precision.

Frequency reduction (F4). Rather than compress each all-reduce, perform fewer of them. Local SGD [Stich, 2019] runs τ inner SGD steps locally between all-reduces; DiLoCo [Douillard et al., 2023a] adds an outer Nesterov optimizer and is empirically stable up to $\tau \approx 500$ for LLM pretraining. Streaming DiLoCo [Douillard et al., 2025] pipelines the outer all-reduce with inner compute, removing the synchronous barrier.

Composition. Frequency reduction multiplies with any per-step compressor. PowerSGD at $\rho_{\text{DP}} = 1/64$ combined with $\tau = 20$ DiLoCo yields an effective $1280\times$ reduction in DP bytes-per-step — enough to clear the threshold in Eq. (13) for our regime.

J.4 Comparison and selection

Table 12 summarizes the practical operating points.

Method	Per-step ratio	Composes with τ	Reported convergence cost
FP8/INT8 quantization	$2\text{--}4\times$	yes	$\leq 1\%$ loss gap
1-bit Adam / signSGD	$32\times$	yes	$1\text{--}3\%$ loss gap
Top- K sparsification	$50\text{--}100\times$	yes	$1\text{--}5\%$ loss gap; index overhead
PowerSGD (rank 4)	$32\text{--}256\times$	yes	$\leq 2\%$ loss gap
Local SGD / DiLoCo	— (τ -based)	—	$\leq 1\%$ loss gap up to $\tau=500$
Streaming DiLoCo	— (τ -based)	—	$\leq$ DiLoCo, hides comm

Table 12: DP-compression methods with demonstrated stability for language-model training. Per-step compression refers to bytes-on-wire relative to a full BF16 all-reduce. Frequency-reduction methods do not have a per-step compression ratio because they reduce the number of all-reduces, not the bytes per all-reduce; they are stacked with F1–F3 rather than competing with them.

The selection rule that follows from Eq. (13) is straightforward:

High bandwidth (≥ 1 Gbps inter-node). The required compression ratio is small, and PowerSGD at $32\text{--}64\times$ alone usually meets it. No frequency reduction needed; implementation is intra-step with no algorithmic changes.

Low bandwidth (~ 200 Mbps). Single-method compression tops out around $64\times$ without convergence loss, leaving roughly an order of magnitude on the table. Composition with DiLoCo ($\tau \geq 10$) is required. Streaming DiLoCo additionally hides the outer all-reduce under inner compute, removing the synchronous barrier — this is our default configuration.

Federated or geo-distributed (≤ 50 Mbps). Composition is mandatory. Streaming DiLoCo with $\tau \in [50, 500]$ stacked with $\rho_{\text{DP}} \leq 1/64$ is the only regime that delivers usable throughput; convergence quality must be re-validated at the chosen τ .

K On the Compatibility with Dual-EMA Optimizers

A natural alternative to our spectral correction is to use the masked gradient as a noisy first-moment signal and reach for a dual-EMA optimizer that is known to tolerate gradient staleness, most notably AdEMAMix [Pagliardini et al., 2024]. AdEMAMix replaces Adam’s single first-moment EMA with a mixture of a *fast* EMA m_1 ($\beta_1 \approx 0.9$) and a *slow* EMA m_2 ($\beta_3 \approx 0.9999$). Its update rule is

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_1^{(t)} + \alpha m_2^{(t)}}{\sqrt{\hat{v}^{(t)} + \epsilon}},$$

where $\hat{m}_1$ is bias-corrected, $\hat{v}$ is Adam’s second moment, and $\alpha \approx 5$ controls the weight of the slow EMA. The empirical finding driving the design is that gradients can remain useful for tens of thousands of steps, and a single EMA cannot simultaneously weight the immediate past and the distant past.

This sounds superficially similar to our two-circuit design, both maintain a slow signal alongside a fast one, so it is reasonable to ask whether AdEMAMix could replace the spectral correction entirely,

1209 with the masked gradient serving as both inputs to the optimizer (the fast and slow EMAs would
 1210 simply be two views of the same noisy signal). We argue this would not work in our setting, for two
 1211 reasons.

1212 **(1) AdEMAMix mixes old gradients, not cleaner gradients.** The slow EMA m_2 in AdEMAMix
 1213 is an exponentially weighted average of the same gradients that feed the fast EMA. Its value comes
 1214 from *temporal smoothing* of a noisy signal, not from any independent high-fidelity source. In our
 1215 setting, the fast circuit’s gradients are *biased away from the true gradient subspace* by approximately
 1216 $\rho\sigma_m^2$ even after spectral filtering (Lemma 2), and unbiased only in the sense of zero conditional
 1217 mean. Smoothing T such gradients with a slow EMA reduces the variance, but does *not* change the
 1218 underlying noise spectrum: directions weakly supported by the signal remain weakly supported in
 1219 the EMA. Our anchor circuit, by contrast, provides an *independent* signal computed from *unmasked*
 1220 activations. Therefore, its principal subspace reflects the true gradient geometry, not a smoothed copy
 1221 of the masked-gradient noise.

1222 **(2) Dual EMAs cannot recover lost subspaces.** A core claim of our analysis (Section E) is that
 1223 the masked gradient is approximately isotropic relative to the anchor basis (Lemma 2, Assumption 6).
 1224 This is why a low-rank spectral filter contracts noise quadratically in rank. AdEMAMix has no
 1225 notion of an anchor basis; it operates entirely in the canonical parameter basis and applies the same
 1226 dual-EMA dynamics elementwise. If the gradient signal lives in a low-rank subspace (as fine-tuning
 1227 gradients do [Aghajanyan et al., 2021, Hu et al., 2022]) and the noise is approximately isotropic,
 1228 AdEMAMix’s elementwise smoothing extracts the signal at rate $O(1/T_{\text{eff}})$ in noise variance, but
 1229 discards the structural advantage of low-rankness. Our spectral filter exploits that structure directly.

1230 However, we do not claim AdEMAMix is unhelpful in our setting. In fact, the two mechanisms are
 1231 largely complementary. A natural composition is to apply AdEMAMix’s dual-EMA dynamics to
 1232 the *filtered* gradient G_t^{proj} rather than to the raw masked gradient. The fast EMA captures recent
 1233 corrected directions, the slow EMA averages the filter’s output over many steps to further reduce
 1234 residual noise, and the spectral correction continues to enforce subspace structure. We did not run
 1235 this composition in the main experiments because our convergence analysis targets the single-EMA
 1236 AdamW baseline directly. We believe this would be an interesting future direction to explore.

1237 NeurIPS Paper Checklist

1238 The checklist is designed to encourage best practices for responsible machine learning research,
1239 addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove
1240 the checklist: **The papers not including the checklist will be desk rejected.** The checklist should
1241 follow the references and follow the (optional) supplemental material. The checklist does NOT count
1242 towards the page limit.

1243 Please read the checklist guidelines carefully for information on how to answer these questions. For
1244 each question in the checklist:

- 1245 • You should answer [Yes], [No], or [N/A].
- 1246 • [N/A] means either that the question is Not Applicable for that particular paper or the
1247 relevant information is Not Available.
- 1248 • Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

1249 **The checklist answers are an integral part of your paper submission.** They are visible to the
1250 reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it
1251 (after eventual revisions) with the final version of your paper, and its final version will be published
1252 with the paper.

1253 The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation.
1254 While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a
1255 proper justification is given (e.g., error bars are not reported because it would be too computationally
1256 expensive” or “we were unable to find the license for the dataset we used”). In general, answering
1257 [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we
1258 acknowledge that the true answer is often more nuanced, so please just use your best judgment and
1259 write a justification to elaborate. All supporting evidence can appear either in the main paper or the
1260 supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification
1261 please point to the section(s) where related material for the question can be found.

1262 IMPORTANT, please:

- 1263 • **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”,**
- 1264 • **Keep the checklist subsection headings, questions/answers and guidelines below.**
- 1265 • **Do not modify the questions and only use the provided macros for your answers.**

1266 1. Claims

1267 Question: Do the main claims made in the abstract and introduction accurately reflect the
1268 paper’s contributions and scope?

1269 Answer: [Yes]

1270 Justification: We have scoped out our contributions accordingly.

1271 Guidelines:

- 1272 • The answer [N/A] means that the abstract and introduction do not include the claims
1273 made in the paper.
- 1274 • The abstract and/or introduction should clearly state the claims made, including the
1275 contributions made in the paper and important assumptions and limitations. A [No] or
1276 [N/A] answer to this question will not be perceived well by the reviewers.
- 1277 • The claims made should match theoretical and experimental results, and reflect how
1278 much the results can be expected to generalize to other settings.
- 1279 • It is fine to include aspirational goals as motivation as long as it is clear that these goals
1280 are not attained by the paper.

1281 2. Limitations

1282 Question: Does the paper discuss the limitations of the work performed by the authors?

1283 Answer: [Yes]

1284 Justification: We provide a limitations section.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: We provide all the necessary assumptions and proofs for our claims.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Full experimental details are provided in the appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: Will be provided upon acceptance.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Comprehensive details are provided in the appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: The experiments are too expensive to obtain error bars.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We have provided the details in the experiments section and the appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- 1441 • The paper should disclose whether the full research project required more compute
1442 than the experiments reported in the paper (e.g., preliminary or failed experiments that
1443 didn't make it into the paper).

1444 **9. Code of ethics**

1445 Question: Does the research conducted in the paper conform, in every respect, with the
1446 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

1447 Answer: [Yes]

1448 Justification: We have followed the code of ethics.

1449 Guidelines:

- 1450 • The answer [N/A] means that the authors have not reviewed the NeurIPS Code of
1451 Ethics.
1452 • If the authors answer [No], they should explain the special circumstances that require a
1453 deviation from the Code of Ethics.
1454 • The authors should make sure to preserve anonymity (e.g., if there is a special consid-
1455 eration due to laws or regulations in their jurisdiction).

1456 **10. Broader impacts**

1457 Question: Does the paper discuss both potential positive societal impacts and negative
1458 societal impacts of the work performed?

1459 Answer: [Yes]

1460 Justification: We have included a discussion in the introduction.

1461 Guidelines:

- 1462 • The answer [N/A] means that there is no societal impact of the work performed.
1463 • If the authors answer [N/A] or [No], they should explain why their work has no societal
1464 impact or why the paper does not address societal impact.
1465 • Examples of negative societal impacts include potential malicious or unintended uses
1466 (e.g., disinformation, generating fake profiles, surveillance), fairness considerations
1467 (e.g., deployment of technologies that could make decisions that unfairly impact specific
1468 groups), privacy considerations, and security considerations.
1469 • The conference expects that many papers will be foundational research and not tied
1470 to particular applications, let alone deployments. However, if there is a direct path to
1471 any negative applications, the authors should point it out. For example, it is legitimate
1472 to point out that an improvement in the quality of generative models could be used to
1473 generate Deepfakes for disinformation. On the other hand, it is not needed to point out
1474 that a generic algorithm for optimizing neural networks could enable people to train
1475 models that generate Deepfakes faster.
1476 • The authors should consider possible harms that could arise when the technology is
1477 being used as intended and functioning correctly, harms that could arise when the
1478 technology is being used as intended but gives incorrect results, and harms following
1479 from (intentional or unintentional) misuse of the technology.
1480 • If there are negative societal impacts, the authors could also discuss possible mitigation
1481 strategies (e.g., gated release of models, providing defenses in addition to attacks,
1482 mechanisms for monitoring misuse, mechanisms to monitor how a system learns from
1483 feedback over time, improving the efficiency and accessibility of ML).

1484 **11. Safeguards**

1485 Question: Does the paper describe safeguards that have been put in place for responsible
1486 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
1487 image generators, or scraped datasets)?

1488 Answer: [N/A]

1489 Justification: Not aware of any specific risk of misuse.

1490 Guidelines:

- 1491 • The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We have done above.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: We do not release any new assets.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: We have not conducted any such experiments.

1543 Guidelines:

1544 • The answer [N/A] means that the paper does not involve crowdsourcing nor research

1545 with human subjects.

1546 • Including this information in the supplemental material is fine, but if the main contribu-

1547 tion of the paper involves human subjects, then as much detail as possible should be

1548 included in the main paper.

1549 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,

1550 or other labor should be paid at least the minimum wage in the country of the data

1551 collector.

1552 **15. Institutional review board (IRB) approvals or equivalent for research with human**

1553 **subjects**

1554 Question: Does the paper describe potential risks incurred by study participants, whether

1555 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)

1556 approvals (or an equivalent approval/review based on the requirements of your country or

1557 institution) were obtained?

1558 Answer: [N/A]

1559 Justification: We have not conducted any such experiments.

1560 Guidelines:

1561 • The answer [N/A] means that the paper does not involve crowdsourcing nor research

1562 with human subjects.

1563 • Depending on the country in which research is conducted, IRB approval (or equivalent)

1564 may be required for any human subjects research. If you obtained IRB approval, you

1565 should clearly state this in the paper.

1566 • We recognize that the procedures for this may vary significantly between institutions

1567 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the

1568 guidelines for their institution.

1569 • For initial submissions, do not include any information that would break anonymity (if

1570 applicable), such as the institution conducting the review.

1571 **16. Declaration of LLM usage**

1572 Question: Does the paper describe the usage of LLMs if it is an important, original, or

1573 non-standard component of the core methods in this research? Note that if the LLM is used

1574 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

1575 scientific rigor, or originality of the research, declaration is not required.

1576 Answer: [No]

1577 Justification: LLMs were only used for writing and editing purposes.

1578 Guidelines:

1579 • The answer [N/A] means that the core method development in this research does not

1580 involve LLMs as any important, original, or non-standard components.

1581 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not

1582 be described.